Recovery and Exposure Windows in an Asynchronous Hybrid KEM Ratchet

Samin Yasar

Independent Researcher
Dhaka, Bangladesh
research@samin-yasar.dev

October 23, 2026

Abstract

Post-compromise recovery in an asynchronous messaging protocol is a timed process, not a binary switch. Fresh randomness, authenticated KEM delivery, and cached out-of-order keys jointly determine how long an exposure persists before a session re-establishes confidentiality. We formalize this interval as an explicit *recovery window* and analyze its interaction with classical-DH exposure in Star-Mesh, an asynchronous hybrid ratchet combining X25519 with ML-KEM-768 and ML-DSA-65.

We introduce: (i) an asynchronous state machine for one-time, authenticated ML-KEM ratchet exchanges handling replay, loss, duplication, and out-of-order arrivals; (ii) a game-based recovery model with explicit delivery, RNG recovery, and erasure assumptions; and (iii) a concrete reduction bounding post-compromise recovery advantage by $\mathbf{Adv}_{\mathrm{PCS}}(\mathcal{A}) \le Q_{\mathrm{sess}} Q_{\mathrm{upd}} \big(Q_{\mathrm{KEM}} \cdot \mathbf{Adv}^{\text{ind-cca2}}_{\mathsf{ML\text{-}KEM}}(\mathcal{B}_{\mathrm{KEM}}) + 2\,\mathbf{Adv}^{\text{euf-cma}}_{\mathsf{ML\text{-}DSA}}(\mathcal{B}_{\mathrm{DSA}})\big) + Q_{\mathrm{KDF}} \cdot \mathbf{Adv}^{\mathrm{prf}}_{\mathsf{HMAC}}(\mathcal{B}_{\mathrm{KDF}}) + \mathbf{Adv}^{\mathrm{prf}}_{\mathrm{BLAKE3}}(\mathcal{B}_{\mathrm{B3}}) + \mathsf{negl}(\lambda)$. Against active adversaries, healing relies on a hardware-isolated identity signing key to authenticate recovery updates against active substitution; against passive-after-compromise adversaries, recovery requires only fresh randomness and state erasure. Before a fresh round-trip completes, intermediate messages remain exposed across Δ messages plus network latency, making the ratchet interval Δ a tunable parameter balancing mixnet Sphinx transport cost against baseline exposure duration. We validate the state machine via machine-checked ProVerif models and a Rust prototype exercising concurrent buffering and bidirectional convergence.

Keywords: Post-Quantum Cryptography; Secure Messaging; Hybrid Ratchet; Recovery Windows; Exposure Windows; ML-KEM; ML-DSA; Forward Secrecy; Post-Compromise Security.

Contents

1 Introduction **3**

2 Preliminaries **5**
- 2.1 Notation . . . 5
- 2.2 Key Encapsulation Mechanisms . . . 7
- 2.3 Digital Signature Schemes . . . 7
- 2.4 Cryptographic Primitives in Star-Mesh . . . 8
- 2.5 Parameter Selection and Security Categories . . . 8

3 The Star-Mesh Protocol **9**
- 3.1 Overview and Trust Model . . . 9
- 3.2 Key Generation and Identity . . . 9
- 3.3 Pre-Key Bundle Publication . . . 10
- 3.4 Hybrid PQ-X3DH Handshake . . . 11
- 3.5 Ephemeral PQ Double Ratchet . . . 13
- 3.6 Prototype Implementations . . . 19

4 Security Model **21**
- 4.1 Cryptographic Assumptions . . . 21
- 4.2 Execution Environment and Adversary . . . 25
- 4.3 Post-Quantum Forward Secrecy (PQ-FS) Game . . . 27
- 4.4 Post-Compromise Security (PCS) Game . . . 27
- 4.5 A Single-Update Recovery Theorem . . . 28

5 Security Analysis **31**
- 5.1 Authentication: Bundle Identity Binding . . . 31
- 5.2 Post-Quantum Forward Secrecy . . . 32
- 5.3 Post-Compromise Security . . . 34

6 Metadata Resistance **35**

7 Bandwidth and Performance Analysis **38**

8 Assumptions and Limitations **41**

9 Related Work **43**

10 Conclusion **45**

A Complete Protocol State Machine and Message Framing **46**
- A.1 Endpoint State Representation . . . 46
- A.2 Canonical Message Framing and Associated Data . . . 47
- A.3 Canonical Asymmetric Update Signatures . . . 48
- A.4 Asynchronous KEM Buffer State Transitions and Canonical Tie-Breaking 48
- A.5 Max-Skip Bound and Staged Commit-After-Authentication . . . 49

References **50**

1 Introduction

The imminent threat of Cryptographically Relevant Quantum Computers (CRQCs) has accelerated the standardization of post-quantum cryptographic (PQC) algorithms by NIST, notably ML-KEM [1] and ML-DSA [2]. While traditional secure messaging protocols such as Signal have begun integrating PQC [3] to defend against "Harvest Now, Decrypt Later" (HNDL) attacks [22], they rely predominantly on centralized servers for key distribution and message routing.

This centralization creates a single point of failure for both availability and metadata privacy: a server compromise can reveal the social graph, timing information, and message volume of all users, even when message contents are encrypted. Decentralized alternatives such as Session [23] and Briar [24] mitigate this architectural risk through onion-routing networks, but they remain committed exclusively to classical elliptic-curve primitives (X25519), leaving them vulnerable to HNDL attacks.

Our Approach. We formulate post-compromise recovery as a *windowed state transition*. Instead of assuming that a successful KEM exchange instantaneously heals a session, the model tracks the interval from clean randomness and authenticated update creation through delivery, ordered application, erasure, and the first testable post-update key. This exposes the precise point at which recovery becomes meaningful and separates it from the messages that remain vulnerable before that point. Star-Mesh instantiates this transition with an asymmetric KEM rather than a commutative DH exchange, combining NIST PQC standards [1, 2] with the classical Double Ratchet structure, while treating the Kademlia DHT and mixnet as deployment context rather than as contributions proved by this paper.

Security boundary. The central theorem concerns one fixed pairwise session and one or more authenticated, deterministically ordered ML-KEM recovery updates. Its freshness predicate requires a target KEM keypair sampled after RNG recovery, an unrevealed recovery identity key, and erasure of the target KEM secret, old root/chain keys, and excluded skipped keys before the target message key is tested. The security model explicitly separates ephemeral session state from the long-term identity signing key: the state compromise query returns volatile runtime state (ephemeral chain/root keys, entropy pool, pending ciphertexts, and cached message keys) but does not reveal the ML-DSA signing key. Against an active adversary, PCS holds strictly under the condition that the identity signing key remains uncompromised (e.g., protected by a hardware security module or secure enclave); if total device expropriation compromises the signing key and the adversary actively tampers with the healing update, recovery is unachievable. (Conversely, against a passive-after-compromise adversary who only eavesdrops post-compromise without injecting, healing succeeds under ML-KEM IND-CCA2, post-t_{rec} RNG freshness, and state erasure without requiring identity key isolation.) The ratchet's PCS adds critical protection beyond static identity keys: static identity keys provide authentication rather than confidentiality, whereas the ratchet continuously refreshes the quantum-resistant symmetric confidentiality root, using the hardware-isolated identity key strictly to prevent active MITM key substitution during healing. Delivery remains fully adversarial: delay, duplication, reordering, modification, and loss are modeled, with an update becoming a confidentiality target only after it completes. This boundary isolates the cryptographic recovery transition from DHT availability, mixnet anonymity, UC composition, side channels, and quantum claims about X25519.

Hybrid Security Rationale. Star-Mesh is a *hybrid* protocol: X25519 remains a classical component, while ML-KEM-768 provides the recovery contribution against a quantum-capable adversary. Accordingly, the paper's quantum-facing claim is deliberately about the post-recovery transition, not every property of the surrounding deployment: X25519 remains classical and ML-DSA carries the quantum-resistant identity binding. A fresh ML-KEM exchange and its authenticated delivery are the mechanism by which the session crosses from the exposure window into the recovered state.

Formally, the hybrid shared secret is computed as

$$SS_{\text{hybrid}} = \texttt{0xFF} \parallel SS_{cl} \parallel SS_{PQ1} \parallel SS_{PQ2}$$

passed through HKDF-SHA3-256 with the ciphertexts ct_1, ct_2 and the classical ephemeral public key $EK_{pk,A}$ bound into the KDF input string $info$. The concatenation-combiner analysis of Giacon et al. [19] motivates this construction, but it is not a direct proof of the complete Star-Mesh protocol: the classical DH component, authenticated asynchronous delivery, deletion schedule, and hidden-input KDF composition require the protocol-specific games and assumptions stated below. Binding the transcript (ephemeral key and ciphertexts) is intended to satisfy the combiner's ciphertext-binding precondition and to limit related-ciphertext/re-encapsulation attacks. HKDF is modeled as a PRF in the proof sketches (see Assumption 4.5 in Section 4), while the complete post-recovery composition uses Assumption 4.7.

Trust Assumptions on Infrastructure. Star-Mesh is *decentralized in deployment*, not trust-free. The DHT nodes collectively control availability of key bundles; a Sybil attack can eclipse a user's mailbox. The mixnet requires that fewer than a fraction f of mix nodes are compromised (the Loopix assumption [15]). These operational assumptions are stated explicitly in Section 4.1.

Contributions.

1. **State machine:** An asynchronous ML-KEM ratchet with authenticated identifiers, one-time secret deletion, replay rejection, and explicit handling of loss, duplication, delay, and out-of-order delivery.

2. **Recovery game:** A freshness definition and PCS game that make RNG recovery, delivery completion, deletion, and pre-existing skipped-key exposure explicit.

3. **Exposure analysis:** A conditional proof target for message keys derived after a completed fresh KEM round-trip, together with an explicit characterization of the asymmetric pre-recovery exposure window (Δ messages plus 1 RTT for the initiator, $\approx$ 1.5 RTT for the responder requiring peer transmission, unbounded if the peer goes quiet) prior to recovery completion.

4. **Executable correspondence:** A Rust prototype that exercises the concurrent buffer, authenticated headers, erasure, and bidirectional convergence, together with ProVerif models checking selected symbolic state-machine properties.

Scope and Proof Status. The main result is a concrete, pairwise recovery theorem with explicit freshness and delivery semantics. The additional game-based arguments in Section 5 extend the same construction and are labeled proof sketches where their

reductions are not fully mechanized or tight in a composable framework. A complete UC-style treatment of the protocol is left to future work [31]. To complement the proof-sketch arguments, we provide a machine-checked ProVerif [33] model (`formal/pq_x3dh.pv` and `formal/ratchet.pv` in the repository) that verifies, in the symbolic Dolev-Yao model, the following properties: session-key secrecy under long-term key reveal, MITM resistance via bundle-signature authentication, pre-compromise forward secrecy, PCS healing after one complete PQ round-trip, and the skipped-key cache caveat (Corollary 5.5, Caveat 2). These are supplementary symbolic checks and do not replace the computational reductions; they provide independent evidence of the protocol logic against a symbolic adversary.

UC formulation status. The repository now specifies a candidate ideal functionality $\mathcal{F}_{\mathrm{SM}}$ in `docs/uc-formulation.md`. It exposes registration, asynchronous delivery, ratchet epochs, state compromise, recovery, and an explicit mixnet leakage interface rather than assuming perfect traffic-flow hiding. This is a proof target and not a theorem: a UC realization claim requires a real/ideal interface, an environment and simulator, composable corruption semantics, and a computational indistinguishability proof. In particular, the existing pairwise games and ProVerif models do not establish UC security.

Research Positioning. Star-Mesh should be viewed as a protocol-design and analysis contribution rather than a production-ready messaging system. The present manuscript focuses on architectural synthesis, adversarial modeling, and hybrid post-quantum ratcheting semantics in a preprint setting, where the goal is to surface a precise design and a clear security model for peer review. A production deployment would additionally require extensive implementation hardening, empirical evaluation, interoperability testing, and operational abuse-resistance analysis.

Paper Organization. Section 2 introduces notation and primitives. Section 3 specifies the Star-Mesh protocol. Section 4 gives formal security definitions and assumptions. Section 5 presents the security claims and proof sketches. Section 6 analyses metadata resistance. Section 7 provides a bandwidth and performance analysis. Section 8 consolidates protocol assumptions and operational limitations. Section 9 surveys related work and Section 10 concludes.

2 Preliminaries

2.1 Notation

We write λ for the security parameter. A function $f(\lambda)$ is *negligible* (written $f \in \mathsf{negl}(\lambda)$) if for every polynomial p, $f(\lambda) < 1/p(\lambda)$ for all sufficiently large λ. We write $x \xleftarrow{\$} S$ for sampling x uniformly at random from a finite set S, and $x \xleftarrow{\$} \mathcal{D}$ for sampling from a distribution $\mathcal{D}$. We denote by $\|$ the concatenation of byte strings.

All query bounds below are functions of λ and are polynomially bounded. For an adversary $\mathcal{A}$, let Q_{sess} bound the number of candidate target-session selections, Q_{upd} bound authenticated recovery-update attempts per session, Q_{KEM} bound the number of KEM challenge embeddings or challenge-relevant encapsulations considered by a reduction, and Q_{KDF} bound the number of post-recovery KDF outputs replaced in a hybrid. These are upper bounds supplied to the reduction; they are not protocol constants and are not additional security assumptions. The symbol $\mathsf{negl}(\lambda)$ absorbs correctness and encoding-collision terms under the stated implementation model. ML-KEM shared secrets and

message keys have 256 bits in the concrete instantiation, but the asymptotic security parameter remains λ.

We write $\mathsf{len}_{32}(s)$ for the *fixed-width, 4-byte big-endian* encoding of the byte length of string s. Using a fixed-width encoding prevents length-extension ambiguity when multiple variable-length fields are concatenated.

For key derivation, $\mathsf{HKDF}(ikm, salt, info, \ell)$ denotes HKDF-SHA3-256 with input key material ikm, optional $salt$, context $info$, and output length ℓ bytes. SHA3-256 is the underlying hash function; it is *not* modeled as a random oracle in this work; rather HKDF is modeled as a two-stage extract-then-expand PRF following the HKDF construction in RFC 5869 [27] and its formal analysis by Krawczyk (CRYPTO 2010) [28] (see Assumption 4.5 in Section 4).

Definition 2.1 (Dual-PRF / Extractor KDF Advantage)**.** *Let $F(ikm, salt, info, \ell)$ be an extract-then-expand KDF (e.g., HKDF-SHA3-256). The* ***one-uniform-component extractor game*** *models settings where the input key material ikm is formed by concatenating $m \geq 1$ components,*

$$ikm = c_1 \,\|\, c_2 \,\|\, \ldots \,\|\, c_m,$$

such that at least one designated component $c_j = u \in \{0,1\}^k$ is computationally uniform and hidden from the adversary, while the remaining components $p = (c_i)_{i \neq j}$ (which may include fixed domain-separation constants, adversarially known classical DH secrets, or pre-key shared secrets) are known to or chosen by the adversary. The game proceeds as follows:

1. *The challenger receives the adversary's choice of public/adversarial components p and context $(salt, info)$. The salt may be public/fixed (e.g. $salt = \mathbf{0}^{32}$ in the initial handshake) or secret/fresh (e.g. the entropy pool EP' or root key RK in ratchet updates).*

2. *The challenger draws an independent uniform hidden component $u \xleftarrow{\$} \{0,1\}^k$, constructs ikm with $c_j = u$ and $c_{i \neq j} = p_i$, computes $y_0 = F(ikm,\ salt,\ info,\ \ell)$, and draws an independent uniform string $y_1 \xleftarrow{\$} \{0,1\}^\ell$. It samples $b \xleftarrow{\$} \{0,1\}$ and returns $(y_b,\ p,\ salt,\ info)$ to the distinguisher $\mathcal{C}$.*

3. *$\mathcal{C}$ outputs a guess $b' \in \{0,1\}$.*

The advantage is $\mathbf{Adv}^{\mathrm{hi}}_F(\mathcal{C}) := |\Pr[b' = b] - \frac{1}{2}|$.

Justification of one-uniform-component sufficiency. *HKDF computes* $\mathsf{PRK} = \mathsf{HMAC}_{salt}(ikm)$ *during extraction and expands it via* $\mathsf{HKDF\text{-}Expand}(\mathsf{PRK}, info, \ell)$*. Following Krawczyk's dual-PRF analysis [28]:*

- ***Public/zero salt (handshake):*** *When $salt = \mathbf{0}^{32}$ is a fixed public constant, HMAC acts as a PRF keyed by its input data (the dual-PRF property). Because u is computationally uniform and statistically independent of p, u provides the secret keying material that ensures the extracted* PRK *is computationally indistinguishable from uniform, despite the known prefix or suffix p.*

- ***Secret/fresh salt (ratchet):*** *When salt is secret and high-entropy (e.g. EP' derived from preceding ratchet epochs), HMAC acts as a standard PRF keyed by salt, and extraction holds* a fortiori *with dual-source entropy from both salt and u.*

- ***Expansion:*** *The expand step evaluates HMAC in standard PRF mode keyed by the pseudorandom* PRK*, guaranteeing that output key material* y_0 *is computationally indistinguishable from a uniform bitstring of length* ℓ*.*

In security reductions, replacing the target shared secret (e.g. SS_{PQ2} *in the handshake, or* SS_{PQ} *in the ratchet) with a uniform string* u *via KEM IND-CCA2 satisfies the input requirement. The* $\mathbf{Adv}^{\mathrm{hi}}_{F}$ *term isolates KDF extraction/expansion security from KEM IND-CCA2 hardness, preventing circularity and double-counting.*

2.2 Key Encapsulation Mechanisms

Definition 2.2 (KEM)**.** *A Key Encapsulation Mechanism* $\Pi = (\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$ *consists of three algorithms:*

- $\mathsf{KeyGen}(1^\lambda) \to (pk, sk)$*: Generates a public/secret key pair.*
- $\mathsf{Encaps}(pk) \to (ct, ss)$*: Encapsulates, producing ciphertext* ct *and shared secret* ss*.*
- $\mathsf{Decaps}(sk, ct) \to ss$*: Decapsulates, recovering* ss *(or* $\perp$ *on failure).*

Definition 2.3 (IND-CCA2 KEM Security)**.** *Let* $\Pi = (\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$ *be a KEM. The advantage of adversary* $\mathcal{A}$ *in the IND-CCA2 game is defined as:*

$$\mathbf{Adv}^{\text{ind-cca2}}_{\Pi}(\mathcal{A}) \coloneqq \left| \Pr[\mathbf{Game}^{\text{ind-cca2}}_{\Pi,\mathcal{A}}(\lambda) = 1] - \frac{1}{2} \right|,$$

where the experiment $\mathbf{Game}^{\text{ind-cca2}}_{\Pi,\mathcal{A}}(\lambda)$ *proceeds as follows:*

1. $(pk, sk) \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$*;*
2. $(ct^*, ss_0) \xleftarrow{\$} \mathsf{Encaps}(pk)$*;* $ss_1 \xleftarrow{\$} \{0,1\}^{|ss_0|}$*;* $b \xleftarrow{\$} \{0,1\}$*;*
3. $b' \xleftarrow{\$} \mathcal{A}^{\mathsf{Decaps}(sk,\cdot)}(pk, ct^*, ss_b)$*;*
4. *The experiment outputs* 1 *if* $b' = b$ *and* $\mathcal{A}$ *never queried* $\mathsf{Decaps}(sk, ct^*)$*; otherwise, it outputs* 0*.*

We say Π *is IND-CCA2 secure if* $\mathbf{Adv}^{\text{ind-cca2}}_{\Pi}(\mathcal{A}) \in \mathsf{negl}(\lambda)$ *for all PPT* $\mathcal{A}$*.*

2.3 Digital Signature Schemes

Definition 2.4 (EUF-CMA Security)**.** *A signature scheme* $\Sigma = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *is Existentially Unforgeable under Chosen-Message Attacks (EUF-CMA) if for all PPT adversaries* $\mathcal{A}$*:*

$$\mathbf{Adv}^{\text{euf-cma}}_{\Sigma}(\mathcal{A}) \coloneqq \Pr\left[\begin{array}{l} (vk, sk) \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda); \\ (m^*, \sigma^*) \xleftarrow{\$} \mathcal{A}^{\mathsf{Sign}(sk,\cdot)}(vk) : \\ \quad \mathsf{Verify}(vk, m^*, \sigma^*) = 1 \\ \quad \wedge\ m^* \notin \mathcal{Q} \end{array}\right] \in \mathsf{negl}(\lambda)$$

where $\mathcal{Q}$ *is the set of queries made to the signing oracle.*

2.4 Cryptographic Primitives in Star-Mesh

Star-Mesh instantiates the following standardized primitives:

- **Identity Signatures:** ML-DSA-65 (Dilithium3) [2], a lattice-based signature scheme whose security reduces to the hardness of Module-LWE and Module-SIS.
- **Key Encapsulation:** ML-KEM-768 (Kyber768) [1], an IND-CCA2 KEM whose security reduces to Module-LWE with parameters $n = 256$, $k = 3$. Public key size: 1184 bytes; ciphertext size: 1088 bytes; shared secret: 32 bytes.
- **Classical DH:** X25519 [26], a Diffie–Hellman function over Curve25519. Key and output size: 32 bytes each. X25519 is vulnerable to a CRQC; it is retained for hybrid security against classical adversaries only.
- **Key Derivation:** HKDF-SHA3-256 [27] for all key derivation steps (see Assumption 4.5); BLAKE3 [29] for message key derivation within the symmetric ratchet.
- **AEAD Encryption:** AES-256-GCM [30] with 96-bit nonces and 128-bit tags.

2.5 Parameter Selection and Security Categories

To establish concrete security levels and estimate protocol overhead, Star-Mesh instantiates FIPS 203 and FIPS 204 parameters aligned with NIST Security Category 3 (providing key-search security equivalent to AES-192). Specifically, we select ML-KEM-768 for post-quantum key encapsulation and ML-DSA-65 for identity signatures. This selection represents a deliberate balance of security, communication bandwidth, and computational cost under decentralized mixnet constraints:

- **Comparison with Category 1 (ML-KEM-512 and ML-DSA-44):** While Category 1 primitives provide smaller public key sizes (e.g., 800 B for ML-KEM-512) and marginally faster execution times, they only offer AES-128 equivalent security. We opt for Category 3 to align with standard high-security targets (AES-192/256) and to provide a wider margin of safety against future algebraic/structural advances in lattice attacks.
- **Comparison with Category 5 (ML-KEM-1024 and ML-DSA-87):** Category 5 provides the highest standardized security (AES-256 equivalent) but introduces substantial bandwidth penalties. ML-DSA-87 signatures exceed 4.5 KB, and ML-KEM-1024 public keys (1,568 B) exceed a standard 1,500-byte MTU by themselves. In a decentralized mixnet architecture like Loopix/Sphinx, messages are routed in constant-sized packets (MTUs) to prevent traffic analysis. Under Category 5, periodic signed updates would require fragment trains of 6 or more packets, severely degrading queue throughput.

Thus, Category 3 (ML-KEM-768 and ML-DSA-65) is selected as the default parameter set for Star-Mesh. The resulting packetization discipline distinguishes routine conversational messages from periodic authenticated ratchet updates:

1. **Intermediate Conversational Messages Fit Under MTU:** Routine messages ($\Delta - 1$ out of every Δ messages) carry only 32-byte classical X25519 ratchet headers and application payloads (192 B payload + 32 B header + 96 B Sphinx framing = 320 B). These fit comfortably within a standard 1,500-byte Ethernet/Sphinx MTU without fragmentation.

2. **Periodic Signed Ratchet Updates Fragment into 4-Packet Trains:** Because state compromise is silent and undetectable, achieving post-compromise security (Theorem 4.20) mandates that *every* periodic KEM update carries mutual ML-DSA-65 signatures (σ_{init} and σ_{resp}, 3,309 bytes each). The initiator payload (1,184 B ML-KEM pk + 3,309 B signature + 192 B application payload = 4,685 B) and responder reply (1,088 B ML-KEM ct + 3,309 B signature + 192 B application payload = 4,589 B) exceed 1,500 B and are fragmented by the transport layer into trains of 4 Sphinx packets per update direction.

3. **Handshake and Pre-Key Bundle Fragmentation:** Initial handshake messages (M_0 = 7,565 B, consisting of 7,533 B raw cryptographic payload plus 32 B framing/identifiers, incorporating the 3,309-byte ML-DSA-65 initiator signature σ_0; or 6,477 B in the SPK-only fallback, 6,445 B raw) and signed DHT pre-key bundles (including 3,309-byte ML-DSA-65 signatures) also exceed 1,500 B. With a per-packet Sphinx payload capacity of 1,404 B (1,500-byte MTU minus 96-byte Sphinx header), transmitting M_0 requires a 6-packet Sphinx train ($\lceil 7{,}565/1{,}404 \rceil = 6$, or 5 packets for SPK-only fallback: $\lceil 6{,}477/1{,}404 \rceil = 5$) or asynchronous DHT mailbox deposits.

3 The Star-Mesh Protocol

3.1 Overview and Trust Model

Star-Mesh is a decentralized protocol deployed without central servers; all key-distribution operations are performed over a Kademlia DHT [13]. Users are identified by their ML-DSA-65 public key IK^{DSA}. Out-of-band verification (e.g., key fingerprint comparison, QR codes) is assumed for initial identity binding, following standard practice in secure messaging [8].

DHT trust assumption. The DHT stores and serves Pre-Key Bundles but is *not* trusted for their authenticity: all bundles are signed under the owner's IK_{sk}^{DSA} and verified by recipients. However, DHT nodes collectively control availability; a Sybil attack can eclipse a user's mailbox. We assume the DHT is Sybil-resistant as an operational assumption (see Section 4.1).

Mixnet trust assumption. Message delivery is routed through a mixnet overlay. We assume at most a fraction $f < 1/2$ of mix nodes are adversarially controlled, following the Loopix threat model [15].

3.2 Key Generation and Identity

Each node derives its cryptographic material from a 256-bit master seed $\mathsf{seed} \xleftarrow{\$} \{0,1\}^{256}$ via deterministic key generation. A 256-bit seed aligns the root-of-trust entropy with the Category 3 security margin targeted by the ML-KEM-768 and ML-DSA-65 parameter choices (Section 2.5). A node's *Identity Bundle* comprises:

- $IK^{DSA} = (IK_{pk}^{DSA}, IK_{sk}^{DSA}) \xleftarrow{\$} \mathsf{ML\text{-}DSA.KeyGen}(1^\lambda)$: the long-term identity signing keypair.
- $IK^{DH} = (IK_{pk}^{DH}, IK_{sk}^{DH}) \xleftarrow{\$} X25519.\mathsf{KeyGen}$: the long-term classical DH keypair.
- $SPK = (SPK_{pk}, SPK_{sk}) \xleftarrow{\$} X25519.\mathsf{KeyGen}$: a semi-static signed pre-key, rotated periodically (e.g., weekly).

- $PQ_SPK = (PQ_SPK_{pk}, PQ_SPK_{sk}) \xleftarrow{\$} \mathsf{ML\text{-}KEM.KeyGen}(1^\lambda)$: a semi-static ML-KEM pre-key, rotated after a bounded number of uses.
- $\{OTPK_i\}_{i=1}^n$: a batch of n one-time X25519 pre-keys.
- $\{PQ_OTPK_j\}_{j=1}^m$: a batch of m one-time ML-KEM pre-keys. Consumption of at least one PQ_OTPK is *mandatory* when available to guarantee initial PQ forward secrecy (see Section 5.1). When the batch is exhausted, only PQ_SPK is used; in that case, the session achieves *session-level* PQ-FS (the pre-key is ephemeral per session) rather than *one-time pre-key PQ-FS* (the pre-key is used exactly once, then zeroized). See Remark 3.1 for the precise distinction.

3.3 Pre-Key Bundle Publication

Node A publishes the following signed bundle to the DHT, keyed by $H(IK_{pk,A}^{DSA})$:

$$\mathsf{Bundle}_A = (IK_{pk}^{DSA}, IK_{pk}^{DH}, SPK_{pk}, PQ_SPK_{pk}, \{OTPK_{pk,i}\}_i, \{PQ_OTPK_{pk,j}\}_j, \sigma)$$

The signature σ is computed over *all* public pre-key material to prevent substitution of individual pre-keys by a DHT adversary:

$$\begin{aligned}\mathsf{BundlePayload} = \texttt{"StarMesh-Bundle-v1"} &\parallel IK_{pk}^{DSA} \parallel IK_{pk}^{DH} \parallel SPK_{pk} \parallel PQ_SPK_{pk} \\ &\parallel \mathsf{len}_{32}(\Vert_i\, OTPK_{pk,i}) \parallel H\left(\Vert_i\, OTPK_{pk,i}\right) \\ &\parallel \mathsf{len}_{32}(\Vert_j\, PQ_OTPK_{pk,j}) \parallel H\left(\Vert_j\, PQ_OTPK_{pk,j}\right) \\ \sigma = \mathsf{ML\text{-}DSA.Sign}&(IK_{sk}^{DSA},\ \mathsf{BundlePayload})\end{aligned}$$

where H is SHA3-256 and $\Vert_i\, X_i = X_1 \| X_2 \| \cdots$ denotes the ordered concatenation of all keys in the batch (not XOR; we use $\|$ throughout for concatenation). Signing the hashes of the full OTPK and PQ-OTPK batches (rather than individual keys) binds the entire batch to σ without inflating the signed payload size; an adversary cannot substitute any identity key, pre-key, or individual OTPK/PQ-OTPK without invalidating σ. A fetcher must also reject a record unless its DHT lookup key equals $H(IK_{pk}^{DSA})$ from the signed bundle; the signing key is therefore not chosen solely from an attacker-controlled record.

Any node fetching Bundle_A verifies σ against the reconstructed $\mathsf{BundlePayload}$ and checks the lookup-key equality before use.

Remark 3.1. **Session-level vs. one-time pre-key PQ-FS.** When a PQ-OTPK is consumed, its secret is zeroized immediately after a single decapsulation; subsequent sessions receive a distinct one-time pre-key. This gives *one-time pre-key PQ-FS* for the handshake: even revealing the long-term IK_{sk}^{DH} and PQ_SPK_{sk} after the fact does not expose the session's shared secret, because PQ_OTPK_{sk} is gone.

When the OTPK batch is exhausted, the initiator encapsulates only against PQ_SPK. Because PQ_SPK is reused across multiple sessions before rotation, an adversary that obtains PQ_SPK_{sk} can retroactively recover the shared secret of all sessions that used it. We term this *session-level* PQ-FS: forward secrecy holds at the granularity of PQ_SPK rotation epochs, not per session. Implementations should replenish OTPK batches proactively to minimize exposure in this mode.

3.4 Hybrid PQ-X3DH Handshake

Alice (initiator) fetches Bob's bundle from the DHT, verifies σ_B over $\mathsf{BundlePayload}_B$, and executes the handshake. The design extends Signal's X3DH [8] with parallel ML-KEM encapsulations.

Step 1 — Classical X3DH Component. Alice generates a fresh ephemeral keypair $EK_A \xleftarrow{\$} X25519.\mathsf{KeyGen}$ and computes:

$$
\begin{aligned}
DH_1 &= \mathsf{DH}(IK^{DH}_{sk,A},\ SPK_{pk,B}) \\
DH_2 &= \mathsf{DH}(EK_{sk,A},\ IK^{DH}_{pk,B}) \\
DH_3 &= \mathsf{DH}(EK_{sk,A},\ SPK_{pk,B}) \\
DH_4 &= \mathsf{DH}(EK_{sk,A},\ OTPK_{pk,B}) \quad \text{(if available; omitted otherwise)}
\end{aligned}
$$

The classical shared secret is $SS_{cl} = DH_1 \| DH_2 \| DH_3 \| DH_4$.

Step 2 — Post-Quantum KEM Component. Alice encapsulates against Bob's ML-KEM pre-keys:

$$
\begin{aligned}
(ct_1,\ SS_{PQ1}) &\xleftarrow{\$} \mathsf{ML\text{-}KEM.Encaps}(PQ_SPK_{pk,B}) \\
(ct_2,\ SS_{PQ2}) &\xleftarrow{\$} \mathsf{ML\text{-}KEM.Encaps}(PQ_OTPK_{pk,B}) \quad \text{(if available)}
\end{aligned}
$$

Step 3 — Cryptographic Binding and Key Derivation. The Associated Data binds the handshake to both parties' post-quantum signing identities, preventing identity misbinding and Unknown Key-Share (UKS)-class attacks (originally demonstrated for STS) [21]:

$$
AD = \texttt{"StarMesh-v1"} \| \mathsf{len}_{32}(IK^{DSA}_{pk,A}) \| IK^{DSA}_{pk,A} \| \mathsf{len}_{32}(IK^{DSA}_{pk,B}) \| IK^{DSA}_{pk,B}
$$

To satisfy the IND-CCA2 security precondition of the concatenation/split-key combiner proved by Giacon et al. [19], the KEM ciphertexts and the classical ephemeral public key are bound into the HKDF input. The binder string $info$ is constructed by appending the length-prefixed transcript parameters to AD:

$$
info = AD \| \mathsf{len}_{32}(EK_{pk,A}) \| EK_{pk,A} \| \mathsf{len}_{32}(ct_1) \| ct_1 \| \mathsf{len}_{32}(ct_2) \| ct_2
$$

The hybrid input key material is formed by combining classical and post-quantum secrets with a fixed one-byte version and format marker `0xFF`:

$$
SS_{\text{hybrid}} = \texttt{0xFF} \| SS_{cl} \| SS_{PQ1} \| SS_{PQ2}
$$

Unlike Signal's 32-byte $\mathbf{F}$ constant [3], this one-byte prefix has no XEdDSA role and carries no cryptographic security weight, which is provided by $info$ with the versioned label `"StarMesh-v1"`. The 64-byte Output Keying Material is then:

$$
OKM = \mathsf{HKDF}\Big(ikm = SS_{\text{hybrid}},\ salt = \mathbf{0}^{32},\ info = info,\ \ell = 64\Big)
$$

Rationale for $salt = \mathbf{0}^{32}$ at initialization. RFC 5869 [27] states that when no truly independent salt is available (as is the case at the start of a fresh handshake where no prior shared state exists), a zero salt is acceptable: the HKDF extract step still provides entropy

concentration from the IKM. In contrast, subsequent post-quantum ratchet steps use $salt = SS_{PQ}$ (the uniform, fresh ML-KEM shared secret), which is a secret, independent input that provides direct standard-model PRF extraction. The different salt usage in the handshake vs. the ratchet is therefore intentional and semantically justified.

The first 32 bytes of OKM initialize the Root Key RK; the second 32 bytes initialize the sending Chain Key CK_s. For explicit key confirmation, Alice derives

$$K_{conf} = \mathsf{HKDF}(OKM,\ \mathbf{0}^{32},\ \texttt{"StarMesh-Confirm"} \,\|\, info,\ 32).$$

Let the consumed pre-key identifiers be:

$$ID_{\text{pre}} \coloneqq \mathsf{len}_{32}(spk_id) \,\|\, spk_id \,\|\, \mathsf{len}_{32}(otpk_id) \,\|\, otpk_id.$$

The core handshake transcript M_0^{core} is defined unambiguously as:

$$\begin{aligned} M_0^{\text{core}} \coloneqq\ & \mathsf{len}_{32}(IK_{pk,A}^{DSA}) \| IK_{pk,A}^{DSA} \| \mathsf{len}_{32}(IK_{pk,B}^{DSA}) \| IK_{pk,B}^{DSA} \| \mathsf{len}_{32}(IK_{pk,A}^{DH}) \| IK_{pk,A}^{DH} \\ & \| \mathsf{len}_{32}(EK_{pk,A}) \| EK_{pk,A} \| \mathsf{len}_{32}(ct_1) \| ct_1 \| \mathsf{len}_{32}(ct_2) \| ct_2 \| ID_{\text{pre}}. \end{aligned}$$

Alice computes $tag_A = \mathsf{HMAC}_{\text{SHA3-256}}(K_{conf}, M_0^{\text{core}})$. Bob verifies tag_A before accepting the session. This gives Bob explicit confirmation that the sender knows the derived secret; it is stronger than an implicit-only statement but is not by itself mutual confirmation.

Handshake Message. Alice signs the core handshake transcript and confirmation tag under her post-quantum identity key:

$$\sigma_0 \xleftarrow{\$} \mathsf{ML\text{-}DSA.Sign}(IK_{sk,A}^{DSA},\ \texttt{"StarMesh-Handshake-M0-Init-v1"} \,\|\, M_0^{\text{core}} \,\|\, tag_A).$$

Alice transmits to Bob:

$$M_0 = (IK_{pk,A}^{DSA},\ IK_{pk,A}^{DH},\ EK_{pk,A},\ ct_1,\ ct_2,\ ID_{\text{pre}},\ tag_A,\ \sigma_0).$$

Bob verifies σ_0 against $IK_{pk,A}^{DSA}$ before proceeding; a failed verification causes M_0 to be rejected. Because σ_0 covers M_0^{core}, it explicitly binds Alice's post-quantum identity to Bob's post-quantum identity ($IK_{pk,B}^{DSA}$), her classical DH identity ($IK_{pk,A}^{DH}$), her ephemeral key ($EK_{pk,A}$), the consumed pre-key identifiers (ID_{pre}), and the ciphertexts. This closes identity-misbinding and cross-recipient replay attacks without requiring Bob to perform a live DHT lookup during message ingestion.

Authentication in M_0. Alice's identity is explicitly bound to the handshake through two complementary mechanisms: (i) the ML-DSA signature σ_0 provides explicit post-quantum initiator authentication before the session key is derived; and (ii) tag_A provides explicit key confirmation from Alice after Bob verifies it. Bob's response includes

$$tag_B = \mathsf{HMAC}_{\text{SHA3-256}}(K_{conf},\ \texttt{"Bob"} \,\|\, M_0^{\text{core}} \,\|\, tag_A),$$

which Alice verifies before treating the handshake as mutually confirmed. The computed SS_{cl} includes $DH_1 = \mathsf{DH}(IK_{sk,A}^{DH}, SPK_{pk,B})$, which is computable only by the holder of $IK_{sk,A}^{DH}$. Together with σ_0, Bob's successful derivation of the same OKM provides both implicit and explicit authentication of Alice's identity, analogously to—and stronger than—the implicit authentication property of standard X3DH [8]. A replay of M_0 with a substituted $EK_{pk,A}$ would fail because the $DH_2/DH_3/DH_4$ components are computed from $EK_{sk,A}$, which the replayer does not possess.

Bob uses the $IK_{pk,A}^{DH}$ included in M_0 together with his own $SPK_{sk,B}$ to recompute DH_1, decapsulates ct_1, ct_2 using the corresponding secret keys, recomputes SS_{hybrid}, and derives the same OKM, completing the handshake.

3.5 Ephemeral PQ Double Ratchet

Following handshake completion, both parties enter the PQ Double Ratchet. The state machine extends the classical Signal Double Ratchet [9] and its generic KEM-based modularization by Alwen, Coretti, and Dodis [10] by interleaving ML-KEM epochs with X25519 DH ratchet steps.

3.5.1 Ratchet State

Each node maintains a session state $\mathcal{S}$:

- $RK \in \{0,1\}^{32}$: Root Key.
- $EP \in \{0,1\}^{32}$: Synchronized entropy pool, initialized from RK and updated only with a successful PQ shared secret.
- $CK_s, CK_r \in \{0,1\}^{32}$: Sending and Receiving Chain Keys.
- $DH_{sk}^{loc}, DH_{pk}^{rem}$: Current local X25519 secret key and remote public key.
- $N_s, N_r \in \mathbb{N}$: Send and receive message counters.
- $PQ_{sk}^{loc}, PQ_{pk}^{loc}$: Current local ephemeral ML-KEM keypair (may be $\perp$).
- $PQ^{outstanding}$: Bounded map from authenticated KEM identifiers to local ephemeral ML-KEM secret keys and their send epochs.
- $PQ^{pending}$: Bounded map from authenticated KEM identifiers to peer ciphertexts awaiting decapsulation.
- $PQ^{retired}$: Set of completed or expired KEM identifiers, used for replay rejection.
- $\mathcal{K}_{skipped}$: Cache of skipped message keys, bounded at capacity 1000.
- $\mathsf{pq_step_counter} \in \{0, \ldots, \Delta\}$: Countdown to the next mandatory PQ ratchet step, with $\Delta = 50$ (matching Apple's PQ3 cadence [7]).

3.5.2 Symmetric Ratchet, Message Framing, and AEAD Header Authentication

Each transmitted message consists of an unencrypted plaintext header h and an AEAD ciphertext c:

$$m_{\text{wire}} = (h,\ c).$$

The header binds the session context, ratcheting parameters, and optional asymmetric ratchet extensions:

$$h = (\mathsf{sid},\ DH_{pk}^{loc},\ N_s,\ PN_s,\ \mathsf{flags},\ \mathsf{ext}),$$

where sid is the 32-byte session identifier, DH_{pk}^{loc} is the sender's current classical DH ratchet public key (32 bytes), N_s is the 32-bit message sequence number in the active sending chain, PN_s is the 32-bit length of the previous sending chain (enabling the receiver to skip keys from past epochs upon a ratchet advance), flags is an 8-bit bitmask indicating asymmetric extension types, and ext contains optional KEM update payloads (such as $(kem_id, PQ_{pk}, \sigma_{\text{init}})$ or $(kem_id, ct, \sigma_{\text{resp}})$ during PQ ratchet rounds).

To encrypt message i in the current sending chain:

$$\begin{aligned} MK_i \| CK'_s &= \mathsf{BLAKE3\text{-}KDF}(CK_s, \texttt{"StarMesh-MK"}) \\ CK_s &\leftarrow CK'_s \quad \text{(then zeroize previous } CK_s\text{)} \\ N_s &\leftarrow N_s + 1 \end{aligned}$$

The header h is strictly authenticated as Associated Data in AES-256-GCM:

$$AD_m := \texttt{"StarMesh-Header-v1"} \,\|\, \mathsf{len}_{32}(h) \,\|\, h,$$

and the plaintext message m is encrypted as

$$c \xleftarrow{\$} \mathsf{AES\text{-}256\text{-}GCM\text{-}Enc}(MK_i,\ \mathsf{nonce} = N_s^{32} \,\|\, \mathbf{0}^{64},\ AD_m,\ m),$$

where N_s^{32} is the 4-byte big-endian encoding of the 32-bit send counter N_s, and $\mathbf{0}^{64}$ is 8 zero bytes, together forming a 12-byte (96-bit) nonce as required by AES-256-GCM. The 32-bit counter field is byte-aligned and caps a single chain at $2^{32} - 1 \approx 4.3 \times 10^9$ messages before nonce reuse; in practice, chains are replaced by DH ratchet steps and PQ ratchet steps well before this bound, but the limit is stated here as a deliberate design choice rather than an implementation artifact. Authenticating h in AD_m guarantees that any tampering with sequence counters, session identifiers, or attached KEM update records causes an immediate GCM authentication failure upon decryption, preventing state desynchronization attacks before live state is modified. MK_i is zeroized immediately after use. We instantiate the symmetric ratchet using BLAKE3-KDF rather than HKDF-SHA3-256 due to the high frequency of symmetric updates (triggered per message). BLAKE3's tree-hashing structure yields significantly lower latency and power consumption on resource-constrained devices, while natively supporting secure domain separation.

3.5.3 DH Ratchet Step

When Alice sends after receiving a message from Bob (i.e., when DH_{pk}^{rem} has been updated), a DH ratchet step is triggered:

$$DH_{\text{new}} = \mathsf{DH}(DH_{sk}^{loc},\ DH_{pk}^{rem})$$

$$RK,\ CK_s = \mathsf{HKDF}(ikm = DH_{\text{new}},\ salt = RK,\ info = \texttt{"StarMesh-RK"},\ \ell = 64)$$

Alice generates a new DH_{sk}^{loc} and zeroizes the previous one. The new DH_{pk}^{loc} is attached to the next outgoing message.

3.5.4 PQ Ratchet Step

The PQ ratchet step is triggered when $\mathsf{pq_step_counter} = 0$ or on the first message of a new session. The step is *role-agnostic*: either party (call them the *PQ-initiator* and the *PQ-responder*) may trigger it when their counter expires. The party whose counter first reaches zero becomes the PQ-initiator for that round. Because state compromise is silent and undetectable by either communicating party, an endpoint cannot know when a compromise has occurred to dynamically request recovery. In standard post-compromise security formulations (e.g., [10, 11]), the adversary is assumed to be passive during the recovery round-trip; under this benign delivery assumption, an unsigned KEM ratchet heals confidentiality (as formalized in Corollary 5.5 and machine-checked in our ProVerif model). However, an adversary holding compromised symmetric ratchet state can mount

an *active impersonation-after-compromise* attack: by injecting forged KEM public keys or ciphertexts authenticated only with compromised symmetric chain keys, an active adversary can indefinitely maintain a man-in-the-middle position and prevent healing. Consequently, to guarantee healing against an *active network adversary post-compromise* (Theorem 4.20), Star-Mesh mandates asymmetric mutual authentication via ML-DSA-65 (*UpdateAuth*) under the endpoints' isolated long-term identity keys on both the public key proposal and the ciphertext response:

1. **Key Generation and Signing (PQ-initiator):** The PQ-initiator generates a fresh ML-KEM keypair $(PQ_{sk}^{loc}, PQ_{pk}^{loc}) \xleftarrow{\$} \mathsf{ML\text{-}KEM.KeyGen}(1^\lambda)$, allocates an identifier kem_id, inserts the pair into $PQ^{outstanding}$, and computes the update signature:

$$m_{\text{init}} \coloneqq \texttt{"UpdateAuth-Init"} \,\|\, sid \,\|\, kem_id \,\|\, PQ_{pk}^{loc} \,\|\, epoch \,\|\, prev,$$
$$\sigma_{\text{init}} \xleftarrow{\$} \mathsf{ML\text{-}DSA.Sign}(IK_{sk,\text{init}}^{DSA},\, m_{\text{init}}).$$

   The initiator attaches $(kem_id, PQ_{pk}^{loc}, \sigma_{\text{init}})$ to the outgoing message.

2. **Verification, Encapsulation, and Staged Transition (PQ-responder):** Upon receiving the message, the PQ-responder looks up kem_id, rejects duplicate or retired identifiers, verifies that *epoch* matches its current sequence counter and *prev* matches the hash of its last accepted transcript (rejecting stale proposals), and verifies σ_{init}:

$$\mathsf{ML\text{-}DSA.Verify}(IK_{pk,\text{init}}^{DSA},\, m_{\text{init}},\, \sigma_{\text{init}}) \stackrel{?}{=} 1.$$

   If verification fails, the payload is rejected. If valid, the responder computes $(PQ_{ct}^{pending}, SS_{PQ}) \xleftarrow{\$} \mathsf{ML\text{-}KEM.Encaps}(PQ_{pk}^{loc})$ and signs the canonical bound response:

$$m_{\text{resp}} \coloneqq \texttt{"UpdateAuth-Resp"} \,\|\, sid \,\|\, kem_id \,\|\, PQ_{pk}^{loc} \,\|\, PQ_{ct}^{pending} \,\|\, epoch \,\|\, prev,$$
$$\sigma_{\text{resp}} \xleftarrow{\$} \mathsf{ML\text{-}DSA.Sign}(IK_{sk,\text{resp}}^{DSA},\, m_{\text{resp}}).$$

   The responder computes the candidate post-quantum root and directional chain update via a single extract and two expands:

$$\begin{aligned} PRK_{\text{staged}} &\leftarrow \mathsf{HKDF\text{-}Extract}(salt = SS_{PQ},\ ikm = RK \,\|\, EP), \\ EP_{\text{staged}} &\leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\text{staged}},\ \texttt{"StarMesh-v1-EntropyPool"} \\ &\qquad \|\, kem_id \,\|\, PQ_{ct}^{pending},\ 32), \\ (RK_{\text{staged}},\, \vec{CK}_{\text{staged}}) &\leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\text{staged}},\ \texttt{"StarMesh-v1-PQ-RK"} \\ &\qquad \|\, kem_id \,\|\, PQ_{ct}^{pending},\ 96), \end{aligned}$$

   where $\vec{CK}_{\text{staged}} = (CK_{A\to B}^{\text{staged}}, CK_{B\to A}^{\text{staged}})$. Because $salt = SS_{PQ}$ is a fresh uniform 32-byte secret under ML-KEM-768 IND-CCA2, extraction is an ordinary PRF step, ensuring pseudorandomness without relying on dual-PRF or public-salt extractor assumptions on SHA3-256. Furthermore, setting $ikm = RK \| EP$ guarantees that past root-key entropy and classical DH-ratchet healing are preserved into the new state. Crucially, the responder does *not* immediately overwrite active root or chain keys. Instead, it buffers the staged update in $\mathsf{PQ}_{\text{staged}}$ and continues to use active CK_r, ensuring initiator messages already in flight under the current chain remain

decryptable. The responder attaches $(kem_id, PQ_{ct}^{pending}, \sigma_{\text{resp}})$ to its reply. Upon receiving the first message authenticated under the new chain $CK_{A\to B}^{\text{staged}}$ (confirming that the initiator received the reply and advanced), the responder commits:

$$RK \leftarrow RK_{\text{staged}}, \quad CK_s \leftarrow CK_{B\to A}^{\text{staged}}, \quad EP \leftarrow EP_{\text{staged}},$$

promotes $CK_r \leftarrow CK_{A\to B}^{\text{staged}}$, retains the prior receiving chain as CK_r^{prev} until trailing skipped messages are resolved, zeroizes intermediate secrets, and resets $\mathsf{pq_step_counter} \leftarrow \Delta$. If the reply is lost and the initiator times out, the initiator retires kem_id without updating RK; upon observing a retransmitted or fresh proposal, the responder clears the uncommitted $\mathsf{PQ}_{\text{staged}}$ entry without modifying RK, preventing desynchronization.

3. **Response Verification and Initiator Advance (PQ-initiator):** Upon receiving the reply, the PQ-initiator looks up kem_id in $PQ^{outstanding}$ and verifies σ_{resp}:

$$\mathsf{ML\text{-}DSA.Verify}(IK_{pk,\text{resp}}^{DSA}, m_{\text{resp}}, \sigma_{\text{resp}}) \stackrel{?}{=} 1.$$

If invalid, the reply is dropped. If valid, the initiator decapsulates

$$SS_{PQ} = \mathsf{ML\text{-}KEM.Decaps}(PQ_{sk}^{loc}, PQ_{ct}^{pending}),$$

and computes the identical single-extract and two-expand derivations:

$$\begin{aligned} PRK' &\leftarrow \mathsf{HKDF\text{-}Extract}(salt = SS_{PQ},\ ikm = RK \,\|\, EP), \\ EP' &\leftarrow \mathsf{HKDF\text{-}Expand}(PRK',\ \texttt{"StarMesh-v1-EntropyPool"} \\ &\qquad \|\, kem_id \,\|\, PQ_{ct}^{pending},\ 32), \\ (RK', \vec{CK}) &\leftarrow \mathsf{HKDF\text{-}Expand}(PRK',\ \texttt{"StarMesh-v1-PQ-RK"} \\ &\qquad \|\, kem_id \,\|\, PQ_{ct}^{pending},\ 96). \end{aligned}$$

The initiator synchronizes its directional chains and root key:

$$RK \leftarrow RK', \quad CK_s \leftarrow CK_{A\to B}, \quad CK_r \leftarrow CK_{B\to A}, \quad EP \leftarrow EP',$$

retaining the prior sending/receiving chain context for in-flight message transitions, applies completed updates in increasing authenticated sequence, retires kem_id, zeroizes PQ_{sk}^{loc}, SS_{PQ}, and old keys, and resets $\mathsf{pq_step_counter} \leftarrow \Delta$. A duplicate response never causes a second root-key update; a timeout retires the identifier without changing the root key and requires a fresh keypair on retry. The complete concurrent-buffer state machine is specified in Appendix A.

Remark 3.2 (Symmetry, Concurrent PQ Steps, and Canonical Tie-Breaking). Both parties maintain independent $\mathsf{pq_step_counter}$ values. It is possible for both counters to reach zero concurrently, prompting both parties to independently emit proposals $U_A = (\mathsf{kem_id}_A, PQ_{pk,A}^{loc}, \sigma_{\text{init},A})$ and $U_B = (\mathsf{kem_id}_B, PQ_{pk,B}^{loc}, \sigma_{\text{init},B})$. Because ML-KEM shared secrets are non-commutative and sequential HKDF evaluations are order-dependent, applying the two updates in arbitrary network arrival order would cause root-key divergence whenever packets interleave differently at the two endpoints. To guarantee deterministic convergence, both endpoints enforce a canonical tie-break rule: the updates are sorted lexicographically by their unique 32-byte authenticated identifiers:

$$U_{(1)} \coloneqq \arg\min_{U\in\{U_A,U_B\}} \mathsf{kem_id}(U), \quad U_{(2)} \coloneqq \arg\max_{U\in\{U_A,U_B\}} \mathsf{kem_id}(U),$$

with ties broken by comparing initiator identity keys IK_{pk}^{DSA}. Both endpoints serialize the derivations in this identical canonical order: $U_{(1)}$ is applied first to (RK, EP) to produce intermediate (RK_1, EP_1), and $U_{(2)}$ is applied second chaining directly from (RK_1, EP_1) to yield the final active root key RK_2 and directional chain keys $\vec{CK}_2$. Regardless of arrival order, both endpoints compute the identical final state. From a security perspective, this sequential chaining preserves post-compromise healing: $U_{(1)}$ applies $\mathsf{HKDF\text{-}Extract}(salt = SS_{PQ,1},\ ikm = RK \,\|\, EP)$ to yield intermediate (RK_1, EP_1); subsequently, $U_{(2)}$ applies $\mathsf{HKDF\text{-}Extract}(salt = SS_{PQ,2},\ ikm = RK_1 \,\|\, EP_1)$. Because both $SS_{PQ,1}$ and $SS_{PQ,2}$ are sampled independently with fresh post-t_{rec} randomness, the second extraction PRF-composes with the first, guaranteeing that $(RK_2, \vec{CK}_2)$ achieves full pseudorandomness under the standard PRF security of HMAC. This compositional structure accounts for the $Q_{\text{KEM}} \in \{1, 2\}$ factor in Theorem 4.20.

Remark 3.3 (Pre-Recovery Exposure Window, Design Trade-offs, and Cadence $\Delta = 50$). The parameter Δ governs the maximal interval between PQ ratchet steps. In normal execution between PQ updates, intermediate message confidentiality against a recording CRQC is preserved because the root key RK incorporates post-quantum secrecy from the preceding handshake or prior PQ ratchet updates, and each classical DH ratchet update evaluates $\mathsf{HKDF}(DH_{\text{new}}, salt = RK, \dots)$. A recording quantum adversary who captures only public ephemeral keys on the wire cannot compute the derived chain keys without knowledge of the secret RK.

Post-compromise exposure bound (informal). Exposure arises when an adversary compromises an endpoint's live session state at time t_{comp}, thereby learning the live root key RK. Against an adversary possessing this compromised state *and* quantum capability (a CRQC), intermediate classical X25519 DH ratchet steps no longer provide healing: the adversary computes DH_{new} by solving the discrete logarithm of the peer's public ephemeral key, and evaluates HKDF with the known RK to track subsequent chain keys. Consequently, healing cannot occur through classical DH steps alone once RK is known; the adversary recovers all messages sent from the point of state compromise until a fresh post-quantum ratchet round-trip *completes*. We note that while the initial handshake is a hybrid construction (combining X25519 and ML-KEM-768), post-compromise healing in the ratchet is *ML-KEM-only* in its cryptographic mechanism: against a CRQC adversary holding compromised state, recovery relies exclusively on the IND-CCA2 security of ML-KEM-768 and mutual ML-DSA-65 update signatures.

The pre-recovery exposure window is asymmetric between the communicating endpoints. From the initiator's perspective, it is bounded by the number of messages sent before: (i) an endpoint emits a fresh PQ_{pk}^{loc} proposal (delayed by up to Δ messages as the ratchet counter decrements), plus (ii) the 1 RTT latency required for the peer's authenticated KEM ciphertext reply to arrive and be accepted. From the responder's perspective, however, the staged commit rule requires buffering candidate keys until an authenticated message (or explicit confirmation ACK) arrives under the new chain; the responder's exposure window is therefore ≈ 1.5 RTT and strictly requires the peer to write. Crucially, if the peer is offline, silent, or network packets are delayed or dropped, this window is unbounded in time until an authenticated message or confirmation ACK successfully completes the staged commit. Only after the full round-trip completes does the session re-enter the secure, fresh state governed by Theorem 4.20.

Design trade-off in the responder commit rule. We emphasize that the responder's extended exposure window (≈ 1.5 RTT, dependent on peer activity) is an explicit architectural trade-off of our staged-commit discipline rather than an inherent theoretical barrier. An alternative commit rule could have the responder commit to the refreshed root

key immediately upon transmitting its ciphertext reply, retaining the prior chain solely for decrypting in-flight messages. While this alternative would shorten the responder's recovery window to 0.5 RTT, a lost reply packet would permanently desynchronize the endpoints: the initiator would eventually timeout and retransmit or propose a new update, while the responder would have already overwritten its root state. Star-Mesh deliberately accepts the longer exposure window to ensure robust, self-healing synchronization in asynchronous, lossy peer-to-peer networks.

Message keys from DH epochs that have already been deleted (enforced deletion schedule, Section 3.5.6) are *not* recoverable: their chain keys were zeroized before the capture event. The exposure is therefore strictly bounded to messages sent *during* the post-compromise window, not the entire session history.

Our choice of $\Delta = 50$ follows the periodic post-quantum ratchet cadence established by Apple's PQ3 protocol [7] (surveyed in [5]). This choice amortises the transmission of post-quantum key material across 50 messages while providing a concrete, tunable baseline on the pre-recovery exposure count. Operators with higher security requirements should reduce Δ; those in high-latency or asynchronous environments must also account for round-trip time and peer availability when estimating the actual exposure window.

3.5.5 Out-of-Order Message Handling: Max-Skip Bound and Staged Commit

When network delivery reorders messages, a message may arrive with sequence counter $N > N_r$:

1. **Max-Skip Bound:** To protect against denial-of-service and memory exhaustion attacks from unauthenticated forged counters, the receiver enforces a strict skip threshold $N_{\text{max_skip}} = 1{,}000$. If $N - N_r > N_{\text{max_skip}}$, the packet is immediately rejected and discarded without evaluating any cryptographic operations or advancing state.

2. **Staged Key Derivation (Commit-After-Authentication):** Rather than immediately advancing the live receiving chain key CK_r, the receiver derives candidate skipped keys into a transient, isolated scratch buffer:

$$MK_j \,\|\, CK_{\text{temp}} = \mathsf{BLAKE3\text{-}KDF}(CK_{\text{temp}}, \texttt{"StarMesh-MK"}),\ j \in [N_r, N-1],$$
$$MK_N \,\|\, CK_{\text{target}} = \mathsf{BLAKE3\text{-}KDF}(CK_{\text{temp}}, \texttt{"StarMesh-MK"}).$$

   The receiver attempts AEAD authentication and decryption using the target candidate key MK_N:

$$m \leftarrow \mathsf{AES\text{-}256\text{-}GCM\text{-}Dec}(MK_N,\ \mathsf{nonce} = N^{32} \,\|\, \mathbf{0}^{64},\ AD_m,\ c).$$

3. **Conditional State Commit:**

   - *If decryption and tag verification succeed:* The receiver commits the staged updates: the skipped keys $\{MK_{N_r}, \ldots, MK_{N-1}\}$ are committed to the bounded skipped-key cache $\mathcal{K}_{skipped}$ (capacity 1,000, oldest keys zeroized upon overflow), the receiving counter advances $N_r \leftarrow N + 1$, and the live chain key updates $CK_r \leftarrow CK_{\text{target}}$. MK_N and intermediate scratch memory are zeroized immediately.
   - *If tag verification fails:* The candidate scratch buffer (including all staged keys and CK_{target}) is instantly zeroized and destroyed. Live state (CK_r, N_r, and

$\mathcal{K}_{skipped}$) remains strictly unmodified, ensuring an attacker cannot desynchronize or destroy live receiver chains by injecting invalid packets with elevated sequence numbers.

4. **Delayed Delivery:** When a delayed message with sequence number $N < N_r$ arrives, the receiver looks up MK_N in $\mathcal{K}_{skipped}$. If present, it verifies and decrypts the payload, extracts m, and immediately zeroizes and deletes MK_N from the cache. If MK_N is absent (already consumed or evicted), the packet is permanently undecryptable and dropped.

3.5.6 Deletion Schedule

The following deletion schedule is mandatory and forms the basis of the forward secrecy and PCS proof sketches. Its enforcement is an *operational assumption* (see Section 4.1); the protocol logic specifies the schedule but cannot itself verify that an implementation honours it.

- **Message Keys (MK_i):** Zeroized immediately after a single AES-GCM encryption or decryption operation.
- **Chain Keys (CK_s, CK_r):** Zeroized immediately upon derivation of the subsequent chain key.
- **Ephemeral X25519 Secrets (DH_{sk}^{loc}):** Zeroized immediately upon generation of the next local DH keypair.
- **Ephemeral ML-KEM Secrets:** Each secret in $PQ^{outstanding}$ is zeroized immediately after its matching ciphertext is decapsulated; outstanding entries are bounded, authenticated, and retired independently.
- **One-Time Pre-Key Secrets:** Zeroized immediately after use in the handshake.
- **Skipped Keys:** Zeroized either upon successful use or upon eviction from the bounded cache.

3.6 Prototype Implementations

To validate the correctness of the cryptographic state machine described in this section, we developed two proof-of-concept implementations available in the open-source repository accompanying this paper[1]:

1. **Python Reference Implementation** (`poc/python/poc.py`): A pure-Python implementation using real post-quantum primitives:
   - `kyber-py` (v1.2, FIPS 203) for ML-KEM-768 [1].
   - `dilithium-py` (v1.5, FIPS 204) for ML-DSA-65 [2], providing the mandatory mutual update signatures (σ_{init} and σ_{resp}).
   - `cryptography` for X25519 (RFC 7748) and AES-256-GCM [26].
   - `hmac` + `hashlib.sha3_256` for HKDF-SHA3-256, split deliberately into one-extract / two-expand calls matching the paper's KDF shape (Assumption 4.7).

[1] https://github.com/Samin-yasar/star-mesh (Apache-2.0 License).

- `blake3` in `derive_key` mode for the symmetric ratchet KDF, producing domain-separated 32-byte and 64-byte outputs.

Public-key and ciphertext sizes produced by this implementation match the paper's tables exactly: ML-KEM-768 yields a 1,184-byte encapsulation key, a 1,088-byte ciphertext, and a 32-byte shared secret; ML-DSA-65 yields a 1,952-byte verifying key and a 3,309-byte signature. The implementation is pure-Python, unaudited, and not constant-time. It is a correct reference artifact for studying message flow, catching KDF-input drift between Python and Rust, and running known-answer tests against the Rust prototype on shared inputs; it must not be used in production.

2. **Rust Prototype** (`poc/rust/src/main.rs`): A high-fidelity implementation using production-ready RustCrypto libraries:

   - `ml-kem` (v0.3) for FIPS 203 ML-KEM-768 [1].
   - `ml-dsa` (v0.1.1) for FIPS 204 ML-DSA-65 [2], providing the mandatory mutual update signatures (σ_{init} and σ_{resp}).
   - `x25519-dalek` (v2.0) for RFC 7748 X25519 [26].
   - `aes-gcm` (v0.10) for AES-256-GCM authenticated payload encryption with header binding.
   - `blake3` (v1.0) and `hkdf` (v0.12) / `sha3` (v0.10) for BLAKE3-KDF and HKDF-SHA3-256.
   - `zeroize` (v1.0) for memory shredding, directly enforcing the deletion schedule specified above.

   The implementation requires a modern Rust compiler ($\geq$ 1.85, with `rust-version = "1.85"` enforced in `Cargo.toml`) due to upstream cryptographic crates supporting the 2024 edition ecosystem. All timing benchmarks cited in Section 7 were produced by this prototype; no performance figures derive from the Python implementation.

Both prototypes execute the full hybrid PQ-X3DH handshake, verify OKM convergence, advance the symmetric and post-quantum ratchet steps, and confirm root-key convergence after a PQ ratchet round-trip, consistent with the PCS healing mechanism argued in Section 5.3.

Formal Symbolic Models. The repository also includes two ProVerif [33] models providing machine-checked symbolic verification in the Dolev-Yao attacker model:

1. `formal/pq_x3dh.pv` (Unbounded Handshake Model): Three queries over unbounded, interleaved sessions:

   - *PQ-FS*: `not attacker(sess_key_witness[])` ✓
   - *MITM resistance*: `event(AliceFinished)` $\implies$ `event(BobPublished)` ✓
   - *Initiator auth*: `event(BobAcceptedM0)` $\implies$ `event(AliceStarted)` ✓

   The third query machine-checks that $IK^{DH}_{pk,A}$ is bound to $IK^{DSA}_{pk,A}$ via σ_0: a DH-key-substitution attacker cannot produce a valid σ_0 without $IK^{DSA}_{sk,A}$.

2. `formal/ratchet.pv` (PQ Double Ratchet Model): Verifies four queries over a bounded execution with state compromise, a PQ ratchet round-trip, and skipped-key cache modeling:

- Pre-compromise forward secrecy: `not attacker(w_pre_comp[])`.
- PCS healing after one PQ round-trip: `not attacker(w_post_heal[])`.
- Skipped-key cache caveat (Corollary 5.5, Caveat 2): `not attacker(w_skipped_pre[])` is false (the key remains exposed post-healing).
- Post-heal message agreement: `event(BobReceivedPostHeal(m))` $\Longrightarrow$ `event(AliceSentPostHeal(m))`.

We clarify the exact threat model of this symbolic model: `formal/ratchet.pv` models a *passive network adversary during the recovery round-trip* (honest message pattern matching on public channels). Under this threat model—matching standard passive-after-compromise PCS definitions—it verifies that the unsigned KEM ratchet logic heals confidentiality and forward secrecy (confirming the passive-case result of Corollary 5.5). It intentionally does not model an active post-compromise adversary mounting man-in-the-middle key substitution, which requires the mutual ML-DSA-65 update signatures proved in Theorem 4.20.

Both models can be reproduced by running `make verify` in the repository root (requires ProVerif 2.04+ installed via OPAM).

4 Security Model

4.1 Cryptographic Assumptions

We state all assumptions explicitly; proof sketches reference only assumptions listed here.

Assumption 4.1 (Module-LWE Hardness and ML-KEM Security)**.** *The Module-LWE problem with parameters $n = 256$, $k = 3$ (ML-KEM-768 parameters) is hard for all QPT adversaries [17]. Under this assumption, ML-KEM-768 is IND-CCA2 secure under the standardized construction [1].*

Assumption 4.2 (EUF-CMA Security of ML-DSA-65)**.** *The ML-DSA-65 signature scheme [2] is Existentially Unforgeable under Adaptive Chosen-Message Attacks (EUF-CMA) for all QPT adversaries.*

Assumption 4.3 (Decisional Diffie–Hellman / PRF-ODH on Curve25519)**.** *The Decisional Diffie–Hellman (DDH) and PRF-ODH problems on the prime-order subgroup of Curve25519 are hard for all classical PPT adversaries (prior to the advent of a Cryptographically Relevant Quantum Computer (CRQC)). Consequently, uncompromised classical ephemeral DH shared secrets are computationally indistinguishable from uniform random group elements.*

Assumption 4.4 (Collision Resistance of SHA3-256)**.** *The cryptographic hash function $H = \text{SHA3-256}$ is collision resistant for all QPT adversaries: for any QPT adversary $\mathcal{A}$, the advantage $\mathbf{Adv}_H^{\text{CR}}(\mathcal{A}) = \Pr[(x, x') \xleftarrow{\$} \mathcal{A}(1^\lambda) : x \neq x' \wedge H(x) = H(x')]$ is negligible in λ.*

Assumption 4.5 (HKDF as Dual-PRF and Extractor)**.** *HKDF-SHA3-256 is modeled as a two-stage extract-then-expand PRF and computational extractor in the standard model, following Krawczyk [28]:*

1. ***Extraction (Dual-PRF property):*** $\mathsf{HMAC}_{salt}(ikm)$ *produces a pseudorandom key* PRK *whenever either (a) salt is a secret, uniform key, or (b) salt is a fixed public string (including salt* $= \mathbf{0}^{32}$*) and ikm contains at least one computationally uniform secret component u independent of known components p (the one-uniform-component extractor property).*

2. ***Expansion (PRF property):*** $\mathsf{HKDF\text{-}Expand}(\mathsf{PRK}, info, \ell)$ *is a standard PRF keyed by* PRK*.*

All proof sketches invoking the pseudorandomness of derived keys rely on this assumption. We do not *model SHA3-256 as a random oracle; standard-model PRF and dual-PRF extraction are sufficient for our reductions.*

Assumption 4.6 (BLAKE3-KDF as PRF)**.** *The keyed hashing mode of BLAKE3 [29], used to derive message keys (MK_i) and next chain keys (CK'_s) from the current chain key, is modeled as a pseudorandom function (PRF) in the standard model. Concretely,* $\mathsf{BLAKE3\text{-}KDF}(k, ctx)$ *with keyed-hash domain separation is PRF-secure: for a uniform key k, the outputs are indistinguishable from uniform bitstrings by any PPT adversary who does not know k. This assumption is required for the pseudorandomness of MK_i used in the AES-256-GCM encryption step, and is load-bearing for the per-message confidentiality claims that underlie both Security Claim 5.2 and Theorem 4.20. The PRF security of BLAKE3's keyed hash mode follows from its design analysis [29] and is consistent with standard-model PRF modeling of keyed sponge and tree-hash constructions.*

Assumption 4.7 (Dual-PRF / One-Uniform-Component Extractor Security — Handshake and Ratchet Cases)**.** *HKDF-SHA3-256 satisfies the partial-uniformity KDF game of Definition 2.1. We state the assumption separately for the two distinct contexts where it is invoked, detailing which component is uniform in each reduction branch.*

Case 1: Handshake (Public Salt, Concatenated Hybrid IKM). *The input key material fed to HKDF in the initial handshake is*

$$ikm = \texttt{0xFF} \,\|\, SS_{cl} \,\|\, SS_{PQ1} \,\|\, SS_{PQ2}$$

(or in the SPK-only fallback, $ikm = \texttt{0xFF} \,\|\, SS_{cl} \,\|\, SS_{PQ1}$). The salt is fixed and public ($salt = \mathbf{0}^{32}$), and the context string $info$ is the length-prefixed public transcript binder ($AD \,\|\, \mathsf{len}_{32}(EK_{pk,A}) \,\|\, EK_{pk,A} \,\|\, \mathsf{len}_{32}(ct_1) \,\|\, ct_1 \,\|\, \mathsf{len}_{32}(ct_2) \,\|\, ct_2$). Under the one-uniform-component extractor assumption, the extracted PRK and derived output key material OKM are computationally indistinguishable from uniform provided that at least one constituent shared secret in ikm is computationally uniform and independent of the remaining components. This covers three distinct reduction branches:

- ***Branch 1a (Post-Quantum FS with OTPK — Primary Reduction):*** *The adversary has compromised classical long-term identity keys (or possesses a CRQC that solves classical discrete logarithms); hence SS_{cl} is known. The semi-static secret SS_{PQ1} may also be known to the adversary. The reduction embeds the IND-CCA2 challenge into the fresh one-time pre-key $PQ_OTPK_{pk,B}$ (ct_2), replacing SS_{PQ2} with a computationally uniform random string $u \xleftarrow{\$} \{0,1\}^{32}$. Here:*

$$u = SS_{PQ2} \quad \textit{(uniform, hidden from } \mathcal{A}\textit{)},$$
$$p = (\texttt{0xFF},\, SS_{cl},\, SS_{PQ1}) \quad \textit{(known or adversarial)}.$$

The one-uniform-component extractor property guarantees that OKM is pseudorandom, with distinguishing advantage bounded by $\mathbf{Adv}^{\text{hi}}_{\text{KDF}}(\mathcal{B}_{\text{KDF}})$. We explicitly acknowledge that in Branch 1a, because salt $= \mathbf{0}^{32}$ is a public constant and the uniform secret $u = SS_{PQ2}$ enters as a suffix of $ikm = p \,\|\, u$, HMAC is evaluated with a fixed key and partially uniform message input. While standard dual-PRF theorems (e.g., [28]) model HMAC when the uniform component occupies the key position, concatenated multi-input extraction under public salt requires treating HMAC as a computational entropy extractor over partially known inputs (Definition 2.1). This is an idealized computational assumption, stronger than standard secret-key PRF security, which we state explicitly to formalize the post-quantum forward secrecy reduction.

- ***Branch 1b (Post-Quantum FS in SPK-Only Fallback):*** *When no OTPK is available in Bob's bundle (Remark 3.1), the handshake omits ct_2 and uses $ikm = \texttt{0xFF} \,\|\, SS_{cl} \,\|\, SS_{PQ1}$. The reduction embeds the IND-CCA2 challenge into $PQ_SPK_{pk,B}$ (ct_1). Here:*

$$u = SS_{PQ1} \quad \textit{(uniform, hidden from } \mathcal{A}\textit{)},$$
$$p = (\texttt{0xFF}, SS_{cl}) \quad \textit{(known or adversarial)}.$$

The one-uniform-component extractor property guarantees pseudorandomness of OKM at the granularity of PQ_SPK rotation epochs.

- ***Branch 1c (Classical Forward Secrecy / Pre-Quantum Adversary):*** *The adversary is pre-quantum (no CRQC), but the post-quantum KEMs are unanalyzed or broken. Under the DDH / PRF-ODH assumption on Curve25519 (Assumption 4.3), the ephemeral DH secret SS_{cl} is computationally pseudorandom and independent of the pre-keys. Here:*

$$u = SS_{cl} \quad \textit{(uniform, hidden from } \mathcal{A}\textit{)},$$
$$p = (\texttt{0xFF}, SS_{PQ1}, SS_{PQ2}) \quad \textit{(known or adversarial)}.$$

The one-uniform-component extractor property guarantees classical forward secrecy against passive and active classical adversaries.

Case 2: Ratchet (Single Extract with Secret Key SS_{PQ}, Two Expands). *In the post-quantum ratchet update (Section 3.5.4), extraction is keyed by the uniform post-quantum shared secret:*

$$PRK = \mathsf{HKDF\text{-}Extract}(salt = SS_{PQ},\ ikm = RK \,\|\, EP),$$

where SS_{PQ} serves as the HMAC key $(salt = SS_{PQ})$ and the concatenated active root key and entropy pool form the input message $(ikm = RK\|EP)$.

- ***Primary reduction branch (PCS Theorem 4.20):*** *In the IND-CCA2 reduction for ML-KEM, the challenge shared secret replaces SS_{PQ} with a computationally uniform string $u \xleftarrow{\$} \{0,1\}^{32}$. Because $salt = SS_{PQ} = u$ is an independent, uniform 32-byte secret key and the input $ikm = RK\|EP$ is known to or tracked by the adversary from t_{comp}, $\mathsf{HMAC}_{SS_{PQ}}(RK\|EP)$ operates as a standard pseudorandom function (PRF) keyed by SS_{PQ}. Pseudorandomness of PRK follows directly from the standard PRF security of HMAC-SHA3-256 (Assumption 4.5, Item 1a), with no dual-PRF, sponge extractor, or hidden-input assumptions required.*

- ***Two expands under distinct domain labels:*** *Given pseudorandom PRK, the two expand evaluations:*

$$EP' = \mathsf{HKDF\text{-}Expand}(PRK,\ \texttt{"StarMesh-v1-EntropyPool"} \\ \|\, kem_id \,\|\, ct,\ 32),$$
$$(RK', \vec{CK}) = \mathsf{HKDF\text{-}Expand}(PRK,\ \texttt{"StarMesh-v1-PQ-RK"} \\ \|\, kem_id \,\|\, ct,\ 96),$$

  produce computationally independent pseudorandom output blocks by the standard PRF property of HKDF-Expand (Assumption 4.5, Item 2). The proof requires only a single replacement of PRK by a uniform string, followed by PRF evaluation of Expand under distinct labels.

- ***Fallback "either component suffices" branch (Classical/KEM-compromise duality):*** *If SS_{PQ} is broken or known to the adversary, but the prior classical state $RK\|EP$ is secret and uncompromised, the extraction step relies on HMAC-SHA3-256 acting as a computational extractor with a known key (salt $= SS_{PQ}$) over the uniform entropy in $RK\|EP$ (dual-PRF/extractor assumption, Assumption 4.5, Item 1b). This dual-PRF assumption is strictly isolated to this fallback robustness claim.*

This formulation ensures that the KDF extraction security is strictly separated from the KEM IND-CCA2 and classical DDH/PRF-ODH hardness reductions across all protocol phases, avoiding circularity or double-counting.

Assumption 4.8 (DHT Availability and Sybil Resistance)**.** *The Kademlia DHT is available and Sybil-resistant in the sense that no adversary can eclipse a target user's mailbox. This is an operational assumption; it is not proven by the protocol logic.*

Assumption 4.9 (Mixnet Honesty Fraction)**.** *At most a fraction $f < 1/2$ of mix nodes are adversarially controlled, following the Loopix threat model [15]. This quantifies the trust requirement on the routing overlay.*

Assumption 4.10 (Deletion Schedule Enforcement)**.** *The deletion schedule specified in Section 3.5 is enforced by the implementation. In particular, message keys, chain keys, and ephemeral secret keys are zeroized at the times specified. This is an operational assumption.*

Assumption 4.11 (RNG Integrity after t_{rec})**.** *The node's random number generator is not compromised by the adversary for any sampling event after time t_{rec}. This models a recovered operating environment (clean OS RNG, side-channel-free execution) following a prior compromise event. Foundational theory for extracting usable randomness from imperfect, seed-dependent sources is given by Dodis et al. [32]; our $\mathcal{O}_{\mathsf{RevealRNG}}$/recovery-time abstraction is an explicit operational modelling assumption of this work. If the RNG is compromised during the PQ ratchet key generation after t_{rec}, PCS is unachievable for that update.*

Assumption 4.12 (Identity Key Isolation across State Compromise)**.** *The endpoint's long-term identity signing key (IK_{sk}^{DSA}) resides in an isolated, tamper-resistant execution environment (such as a hardware security module, secure enclave, or protected operating-system keystore) and is not exfiltrated during an ephemeral session state compromise. The corruption oracle $\mathsf{Corrupt}(P, t)$ returns volatile session memory (ephemeral chain keys, root keys, entropy pool, pending ciphertexts, and transient keys), but strictly excludes IK_{sk}^{DSA}.*

This assumption additionally requires that the adversary cannot query the enclave's signing oracle on the target recovery update transcripts*: in standard EUF-CMA security, an adversary who legitimately obtains a signature on a message from the signing oracle at* any *point in the experiment has not produced a forgery. Therefore, to reduce active impersonation resistance to EUF-CMA, the adversary must not query* $\mathcal{O}^{\text{Sign}}$ *on the target update transcripts (*`"UpdateAuth-Init"` *or* `"UpdateAuth-Resp"`*) at any time during the experiment. If the adversary could query the enclave signing oracle on target update parameters, it could produce validly authenticated updates without breaking EUF-CMA. (Against a passive-after-compromise adversary that reads state but does not inject or obtain signing-oracle queries, healing holds relying solely on KEM IND-CCA2 and a clean RNG without requiring identity signatures, as stated in Corollary 5.5.)*

Post-compromise security against an active adversary holds only if both conditions are met—the identity signing key is not exfiltrated and *the adversary cannot query the enclave's signing oracle on the target recovery update transcript. If either condition fails, an update forging or replay attack becomes possible and active post-compromise recovery cannot be guaranteed.*

Remark 4.13 (Entropy-pool recovery boundary). **Key point:** the entropy pool provides no additional security unless SS_{PQ} itself is unknown to the adversary, i.e., unless the fresh ML-KEM keypair was generated with an uncompromised RNG.

The implementation maintains a synchronized entropy pool EP updated after a successful PQ ratchet via $PRK' \leftarrow \mathsf{HKDF\text{-}Extract}(salt = SS_{PQ},\ ikm = RK \,\|\, EP)$ and $EP' \leftarrow \mathsf{HKDF\text{-}Expand}(PRK',\ \texttt{"StarMesh-v1-EntropyPool"} \,\|\, kem_id \,\|\, ct,\ 32)$. The ciphertext ct and kem_id appear as public transcript context for domain separation, not as an entropy source: a known root key concatenated with a public ciphertext cannot heal a compromised RNG. If the adversary already knows SS_{PQ} (because it compromised the RNG before the fresh keypair was generated), the pool update is completely transparent to the adversary. Recovery therefore depends entirely on whether the ML-KEM key-generation event at t_{rec} was fresh. A stronger theorem under partially subverted RNGs would require an explicit entropy-source model (e.g., following [32]) and is left for future work.

4.2 Execution Environment and Adversary

We model execution using a multi-party protocol execution game adapted from [11, 12]. Oracle naming in this paper is normalized to a single notation ($\mathcal{O}^{Reveal}$, $\mathcal{O}^{RevealLTK}$, etc.); it is intended to be semantically aligned with their state-reveal and long-term-key compromise queries rather than a verbatim reproduction of prior oracle labels. Let $\mathcal{P} = \{P_1, \ldots, P_n\}$ be a set of parties. Each party P_i may hold multiple sessions; we denote by π_i^s the s-th session of party P_i.

We evaluate Star-Mesh against a **quantum-active adversary** $\mathcal{A}_{\text{quant}}$, modeled as a non-uniform quantum polynomial-time (QPT) algorithm equipped with a CRQC. $\mathcal{A}_{\text{quant}}$ has the following capabilities.

Definition 4.14 (Adversarial Oracles)**.** *The adversary* $\mathcal{A}_{\text{quant}}$ *may adaptively query the following oracles:*

- $\mathcal{O}^{Send}(i, s, m)$*: Delivers, injects, modifies, or drops message* m *to/from session* π_i^s*.* $\mathcal{A}_{\text{quant}}$ *has full network control.*
- $\mathcal{O}^{Reveal}(i, s)$*: Returns the current ratchet state* $\mathcal{S}$ *of session* π_i^s *(simulating device compromise).*

- $\mathcal{O}^{RevealLTK}(i)$: *Returns the long-term secret keys of party* P_i.
- $\mathcal{O}^{Sign}(i, m)$: *Invokes the hardware-isolated keystore/enclave of party* P_i *to return an ML-DSA-65 signature on message* m *under* $IK^{DSA}_{sk,i}$.
- $\mathcal{O}^{RevealRNG}(i, t)$: *Reveals or controls the output of* P_i*'s random number generator at time* t. *This models a compromised OS RNG, a side-channel attack, or any mechanism by which the adversary learns or influences fresh randomness [32].*
- $\mathcal{O}^{Test}(i, s, e)$: *Returns either the real epoch-*e *message key* MK_e *or a uniformly random key; the adversary's goal is to distinguish.*

Definition 4.15 (Freshness Predicates: PQ-FS and PCS Freshness)**.** *An epoch of session* π^s_i *is fresh if no* $\mathcal{O}^{Test}$ *query has been made on it, and the adversary's query history does not trivially reveal the target message key. Because forward secrecy and post-compromise security protect disjoint temporal regimes (pre-compromise vs. post-recovery), we distinguish the freshness predicate for each game:*

1. ***PQ-FS Freshness (for Game*** $\mathbf{Game}^{\text{PQ-FS}}$***):*** *Target epoch* e *is fresh if:*
   - $\mathcal{O}^{Reveal}(i, s)$ *(or peer* $\mathcal{O}^{Reveal}(j, s')$*) was either never queried, or was queried strictly after the ratchet step establishing* MK_e *and after all state capable of deriving* MK_e *was erased per the deletion schedule.*
   - *The one-time pre-key secret* $PQ_OTPK_{sk,B}$ *used to establish session* π^s_i *was not revealed prior to its erasure.*
   - $\mathcal{O}^{RevealRNG}$ *was not queried at the time ephemeral secrets establishing epoch* e *were sampled.*
2. ***PCS Freshness (for Game*** $\mathbf{Game}^{\text{PCS}}$***):*** *After a state reveal at* t_{comp}*, target epoch* e' *is fresh if:*
   - e' *occurs strictly after an authenticated PQ ratchet round-trip completes and commits post-*t_{rec}.
   - *No subsequent* $\mathcal{O}^{Reveal}$ *query is issued on session* π^s_i *(or its peer session) between the recovery round-trip and epoch* e'.
   - *The message key* $MK_{e'}$ *is not a cached skipped key* $\mathcal{K}_{skipped}$ *whose derivation chain was exposed prior to* t_{rec}.
   - $\mathcal{O}^{RevealRNG}$ *was not queried at or after* t_{rec} *when the recovery ML-KEM keypair or encapsulation randomness was sampled.*
   - *Signing oracles* $\mathcal{O}^{Sign}(i, \cdot)$ *and* $\mathcal{O}^{Sign}(j, \cdot)$ *were never queried during the experiment on the specific target update transcripts (*`"UpdateAuth-Init"` *or* `"UpdateAuth-Resp"`*) establishing epoch* e' *(EUF-CMA non-triviality).*
   - *The recovery proposal* (PQ_{sk}, PQ_{pk}) *was sampled with clean RNG after* t_{comp} *(not a stale proposal held in memory across compromise or expired under* T_{timeout}*).*

Remark 4.16 (Relationship between Freshness Predicates). Definition 4.15 formalizes the complementary relationship between the two security guarantees. In PQ-FS (Definition 4.17), the challenge epoch e occurs *prior* to any un-erased state reveal, formalizing forward secrecy under backward compromise. In PCS (Definition 4.18), state reveal occurs

prior to the recovery exchange at t_{comp}, and the challenge epoch e' occurs *after* the authenticated round-trip completes with clean randomness post-t_{rec}, formalizing healing after forward compromise. Banning signing queries on target recovery transcripts at any point in the experiment enforces standard EUF-CMA non-triviality, while permitting the adversary full access to $\mathcal{O}^{Sign}$ on all non-target transcripts to model hardware enclave availability.

4.3 Post-Quantum Forward Secrecy (PQ-FS) Game

Definition 4.17 (PQ-FS Security)**.** *Consider the following game* $\mathbf{Game}_{\mathcal{A}}^{\text{PQ-FS}}(\lambda)$*:*

1. *The challenger runs all honest party key generation and initializes the DHT.*
2. $\mathcal{A}_{\text{quant}}$ *adaptively issues* $\mathcal{O}^{Send}$, $\mathcal{O}^{Reveal}$, $\mathcal{O}^{RevealLTK}$, *and* $\mathcal{O}^{RevealRNG}$ *queries.*
3. $\mathcal{A}_{\text{quant}}$ *selects a target session* π_i^s *and a target epoch* e *that is fresh per Definition 4.15. It additionally requires that* e *precedes the time of any* $\mathcal{O}^{Reveal}$ *query on* π_i^s*.*
4. *The challenger samples* $b \xleftarrow{\$} \{0,1\}$*. If* $b = 0$*, it returns* MK_e*; if* $b = 1$*, it returns* $r \xleftarrow{\$} \{0,1\}^{256}$*.*
5. $\mathcal{A}_{\text{quant}}$ *outputs a guess* b'*.*

The advantage is $\mathbf{Adv}_{\text{SM}}^{\text{PQ-FS}}(\mathcal{A}) := |\Pr[b' = b] - \frac{1}{2}|$*. The protocol achieves* ***PQ-FS*** *if this advantage is negligible for all QPT* $\mathcal{A}_{\text{quant}}$*.*

4.4 Post-Compromise Security (PCS) Game

Definition 4.18 (PCS Security)**.** *Consider the following game* $\mathbf{Game}_{\mathcal{A}}^{\text{PCS}}(\lambda)$*:*

1. *The challenger initializes all sessions.* $\mathcal{A}_{\text{quant}}$ *may issue arbitrary oracle queries.*
2. $\mathcal{A}_{\text{quant}}$ *issues* $\mathcal{O}^{Reveal}(i, s)$ *at time* t_{comp}*, obtaining the full ratchet state.*
3. $\mathcal{O}^{RevealRNG}(i, t)$ *may be queried at any time* $t \leq t_{rec}$ *for some recovery time* $t_{rec} \geq t_{comp}$*. After* t_{rec}*, by Assumption 4.11, the RNG is clean; freshness Condition 3 (Definition 4.15) therefore does not apply to keys sampled after* t_{rec}*.*
4. *After time* t_{rec}*, at least one complete authenticated PQ buffer round-trip is executed: Alice samples a fresh keypair after* t_{rec} *with a clean RNG (not replaying a proposal sampled prior to* t_{comp}*), inserts it under a new kem_id, transmits the public key proposal to Bob with* σ_{init}*, Bob returns an authenticated ciphertext ct carrying the same identifier with* σ_{resp}*, and Alice decapsulates it, commits the updated state, retires the identifier, and zeroizes the matching secret.*
5. $\mathcal{A}_{\text{quant}}$ *selects a target epoch* e' *in session* π_i^s *that is fresh per Definition 4.15, satisfying (a)* e' *occurs after the round-trip of step 4 completes and commits; (b) the chain key for epoch* e' *is derived* after *the round-trip (not from a pre-compromise chain key); and (c) the message key* $MK_{e'}$ *is not in* $\mathcal{K}_{skipped}$ *from before* t_{rec}*. The challenger responds with a Test oracle call.*
6. $\mathcal{A}_{\text{quant}}$ *outputs a guess* b'*.*

The protocol achieves ***PCS*** *if* $\mathbf{Adv}_{\text{SM}}^{\text{PCS}}(\mathcal{A}) \in \mathsf{negl}(\lambda)$ *for all QPT* $\mathcal{A}_{\text{quant}}$*.*

Remark 4.19 (Relation between Multi-Party Game and Single-Update Reduction). Definition 4.18 defines post-compromise security in the full multi-session execution environment. Section 4.5 formalizes the concrete pairwise reduction targeting one fresh recovery round-trip using the operational oracle suite (NetSend, Corrupt, RevealLTK, Sign, RevealRNG, RevealKEM, Test), where state exposure $\mathcal{O}^{Reveal}(i, s)$ is instantiated via $\mathsf{Corrupt}(P_i, t)$ and network actions are controlled via NetSend.

4.5 A Single-Update Recovery Theorem

The following theorem deliberately targets one authenticated recovery update rather than the complete decentralized protocol. It makes explicit a condition that is necessary after state compromise: neither the fresh KEM public key nor the responding ciphertext can be authenticated only with a ratchet key that the adversary learned at t_{comp}. In the statement below, *UpdateAuth* consists of mutual ML-DSA signatures by the endpoints' long-term identity keys: the initiator signs

$$\sigma_{\text{init}} \leftarrow \mathsf{ML\text{-}DSA.Sign}\big(IK^{DSA}_{sk,A},\ \texttt{"UpdateAuth-Init"} \,\|\, \mathsf{sid} \,\|\, \mathsf{kem_id} \,\|\, PQ_{pk} \,\|\, \mathsf{epoch} \,\|\, \mathsf{prev}\big),$$

and the responder signs the canonical bound response:

$$\sigma_{\text{resp}} \leftarrow \mathsf{ML\text{-}DSA.Sign}\big(IK^{DSA}_{sk,B},\ \texttt{"UpdateAuth-Resp"} \,\|\, \mathsf{sid} \,\|\, \mathsf{kem_id} \,\|\, PQ_{pk} \,\|\, ct \,\|\, \mathsf{epoch}' \,\|\, \mathsf{prev}'\big),$$

where prev and prev′ are hashes of the respective last accepted ratchet transcripts. Authenticating both the KEM public key proposal and the responding ciphertext ensures that an adversary who possesses Bob's compromised ratchet state cannot answer Alice's *kem_id* with an independently encapsulated ciphertext whose shared secret the adversary knows. A deployment may replace these signatures with another authentication mechanism, but it must provide the same unforgeability guarantee and must not reveal its signing keys in the compromise query.

Oracle semantics. For a fixed target session π with endpoints A and B, the challenger maintains

$$(RK, EP, CK_s, CK_r, \mathsf{outstanding}, \mathsf{retired}, \mathcal{K}_{skipped}, \mathsf{transcript})$$

for each endpoint. The adversary has quantum polynomially many adaptive queries:

- $\mathsf{NetSend}(\pi, x)$: deliver, delay, duplicate, reorder, modify, or drop a protocol record x. A record is accepted only if its authenticated transcript and kem_id checks pass.
- $\mathsf{Corrupt}(P, t)$: return the complete live state of P at time t, including current chain/root keys, outstanding KEM secrets, pending ciphertexts, skipped keys, and all randomness already sampled. Erased values are not returned.
- $\mathsf{RevealLTK}(P)$: return the ML-DSA signing and other long-term secret keys. The target game aborts if the target update's signing key is revealed.
- $\mathsf{Sign}(P, m)$: return an ML-DSA-65 signature on m under party P's long-term identity signing key $IK^{DSA}_{sk,P}$. The target game aborts if $\mathsf{Sign}(P, \cdot)$ is queried on a message prefixing or matching `"UpdateAuth-Init"` or `"UpdateAuth-Resp"` for the target session update at any point during the experiment.

- $\mathsf{RevealRNG}(P, t)$: reveal or control the randomness used at time t. The target game aborts if this query covers generation of the target KEM keypair.
- $\mathsf{RevealKEM}(\mathsf{kem_id})$: return an outstanding KEM secret key. The target game aborts if the identifier is the target identifier.
- $\mathsf{Test}(\pi, e)$: if (π, e) is fresh, return either the real first message key in target post-update epoch e or an independent uniform 256-bit string; otherwise return $\perp$. Only one Test query is allowed.

Freshness and admissible target. The target update $(\mathsf{kem_id}^*, PQ_{pk}^*, ct^*)$ is fresh only if:

1. its KEM keypair is generated after t_{rec} with independent uniform randomness and no $\mathsf{RevealRNG}$ query covers that generation (strictly excluding stale-proposal replays sampled prior to t_{comp});
2. both update messages are authenticated under identity keys not returned by $\mathsf{RevealLTK}$, and neither party's signing oracle was queried via Sign on the target update messages m_{init}^* or m_{resp}^* at any point during the experiment: (a) σ_{init} on $(\texttt{"UpdateAuth-Init"}, \mathsf{sid}, \mathsf{kem_id}^*, PQ_{pk}^*, \mathsf{epoch}, \mathsf{prev})$ verifies under endpoint A's identity key; (b) σ_{resp} on $(\texttt{"UpdateAuth-Resp"}, \mathsf{sid}, \mathsf{kem_id}^*, PQ_{pk}^*, ct^*, \mathsf{epoch}', \mathsf{prev}')$ verifies under endpoint B's identity key; and the signed session, identifier, public key, ciphertext, predecessor hashes, and epochs are exactly those accepted by both endpoints;
3. the matching ciphertext is delivered and decapsulated exactly once, after which the KEM secret, old root/chain keys, and any pre-update skipped keys are erased as specified;
4. no $\mathsf{Corrupt}$ query occurs from target key generation through erasure of the tested message key, and no query reveals the target shared secret; and
5. the tested message key is derived after the completed update and is not in $\mathcal{K}_{skipped}$ from before t_{rec}.

Network queries remain unrestricted: the adversary may delay or drop the update, but a target is not admissible unless the update completes. Thus availability is not being confused with confidentiality.

Theorem 4.20 (Adaptive single-update PCS bound)**.** *Let* Π *be the ratchet state machine with the acceptance and deletion rules above. Let* $\mathcal{A}$ *be any QPT adversary making at most* Q_{sess} *session selections,* Q_{upd} *authenticated update attempts per session,* Q_{KEM} *candidate KEM proposals per update round (*$Q_{\text{KEM}} \in \{1, 2\}$ *in standard Star-Mesh execution), and* Q_{KDF} *post-update KDF evaluations. Assume ML-KEM-768 is IND-CCA2 secure against QPT adversaries, ML-DSA-65 is EUF-CMA secure, HMAC-SHA3-256 and HKDF-Expand are PRF-secure, and BLAKE3-KDF is PRF-secure (Assumption 4.6). If* $\mathcal{A}$ *has advantage* ϵ_{PCS} *in the adaptive game above, then there exist QPT reductions* $\mathcal{B}_{\text{KEM}}$, $\mathcal{B}_{\text{DSA}}$, $\mathcal{B}_{\text{KDF}}$, *and* $\mathcal{B}_{\text{B3}}$ *such that*

$$\epsilon_{\text{PCS}} \le Q_{\text{sess}} Q_{\text{upd}} Q_{\text{KEM}} \, \mathbf{Adv}^{\text{ind-cca2}}_{\mathsf{ML\text{-}KEM}}(\mathcal{B}_{\text{KEM}}) + 2 Q_{\text{sess}} Q_{\text{upd}} \, \mathbf{Adv}^{\text{euf-cma}}_{\mathsf{ML\text{-}DSA}}(\mathcal{B}_{\text{DSA}}) + Q_{\text{KDF}} \, \mathbf{Adv}^{\text{prf}}_{\text{HMAC}}(\mathcal{B}_{\text{KDF}}) + \mathbf{Adv}^{\text{prf}}_{\text{BLAKE3}}(\mathcal{B}_{\text{B3}}) + \mathsf{negl}(\lambda).$$

Consequently, the tested post-update message key is indistinguishable from uniform, subject to the freshness conditions. The bound makes no confidentiality claim for messages sent before completion, for pre-update skipped keys, or for executions in which the adversary continues to control either endpoint's RNG or identity-signing key.

Reduction algorithm. **1. Where PCS comes from (The Salted HMAC Extraction Mechanism):** A symmetric-state compromise at t_{comp} exposes active root key RK, entropy pool EP, and active chain keys CK. Post-compromise healing is achieved precisely because in the post-quantum ratchet derivation:

$$PRK = \mathsf{HKDF\text{-}Extract}(salt = SS_{PQ},\ ikm = RK \,\|\, EP),$$

the uncompromised post-t_{rec} shared secret SS_{PQ} occupies the secret key ($salt$) position of HMAC-SHA3-256. Even though the input message $ikm = RK \,\|\, EP$ is fully known to the adversary from the earlier compromise, HMAC keyed with the uniform secret SS_{PQ} acts as a standard PRF over known messages. This architectural choice—feeding fresh KEM randomness as salt rather than IKM—is the exact mechanism through which root-key pseudorandomness and post-compromise confidentiality are restored.

2. KEM Reduction and Decapsulation Simulation: The KEM reduction $\mathcal{B}_{\text{KEM}}$ guesses a target session (at most Q_{sess}), target update index (at most Q_{upd}), and contributing KEM proposal branch (at most $Q_{\text{KEM}} \in \{1, 2\}$), losing factor $Q_{\text{sess}} Q_{\text{upd}} Q_{\text{KEM}}$. It embeds the IND-CCA2 challenge public key pk^* as the initiator's proposal PQ^*_{pk} and challenge ciphertext ct^* as the responder's reply. For all non-target sessions and updates, $\mathcal{B}_{\text{KEM}}$ generates honest ML-KEM keypairs, decapsulates natively, and answers all oracle queries. For the target update, the simulator handles decapsulation as follows:

- If the adversary faithfully delivers the genuine challenge response ct^*, the simulator substitutes the IND-CCA2 challenge shared secret ss_b directly into the extraction derivation without invoking the decapsulation oracle on ct^*.
- If the adversary injects an adversarial ciphertext $ct' \neq ct^*$, the honest initiator requires a valid signature σ_{resp} under responder Bob's long-term identity key $IK^{DSA}_{pk,B}$ over ct'. Because Bob's signing key is hardware-isolated and uncompromised (Assumption 4.12) and Bob only signed ct^*, generating a valid signature on $ct' \neq ct^*$ constitutes an existential forgery against ML-DSA-65. Thus, any substitute ct' is rejected during signature verification with overwhelming probability, ensuring $\mathcal{B}_{\text{KEM}}$ never encounters an accepted non-challenge ciphertext for which decapsulation is required.

Freshness conditions guarantee that $\mathcal{A}$ cannot query $\mathcal{O}^{Reveal}$ to expose PQ^*_{sk} before erasure. When $b = 0$, ss_0 is the real shared secret; when $b = 1$, ss_1 is uniformly random, transitioning SS_{PQ} to uniform.

3. Authenticated Update Signatures (DSA Reduction): An active adversary attempting to hijack recovery must forge an update signature under an honest party's isolated identity key. In each update round there are two signatures: σ_{init} (signed by initiator A over proposal PQ_{pk}) and σ_{resp} (signed by responder B over response ct). The DSA reduction $\mathcal{B}_{\text{DSA}}$ guesses the target session (loss Q_{sess}), the update index (loss Q_{upd}), and which role's signature is forged (2 choices: initiator proposal vs. responder reply). This yields an exact guessing loss of $2Q_{\text{sess}}Q_{\text{upd}}$. Any accepted update containing modified transcript parameters (sid, kem_id, public key, ciphertext, epoch, or predecessor) without

a query to $\mathcal{O}^{Sign}$ on that specific transcript immediately yields an EUF-CMA forgery against ML-DSA-65.

4. KDF and Message Key Hybrids: The KDF reduction applies Assumption 4.7, **Case 2 (Ratchet)**:

- *Extract PRF Hybrid:* Replacing SS_{PQ} with uniform $u \xleftarrow{\$} \{0,1\}^{32}$ turns $PRK = \mathsf{HMAC}_u(RK \,\|\, EP)$ into a standard PRF evaluation. Across at most Q_{KDF} post-update KDF evaluations in the experiment, a standard hybrid argument across sequential KDF invocations bounds the distinguishing loss by $Q_{\mathrm{KDF}} \cdot \mathbf{Adv}^{\mathrm{prf}}_{\mathrm{HMAC}}(\mathcal{B}_{\mathrm{KDF}})$.
- *Expand PRF Hybrid:* Given pseudorandom PRK, evaluating $\mathsf{HKDF\text{-}Expand}$ under distinct domain labels (`"StarMesh-v1-EntropyPool"` and `"StarMesh-v1-PQ-RK"`) produces computationally independent pseudorandom output blocks for EP', RK', and directional chain key CK_s.
- *Symmetric Ratchet Hybrid:* Finally, tested message key MK_0 is derived from CK_s as $\mathsf{BLAKE3}(CK_s, \texttt{"StarMesh-MK"})$. By the PRF security of BLAKE3-KDF (Assumption 4.6), MK_0 is computationally indistinguishable from uniform random bits, contributing the $\mathbf{Adv}^{\mathrm{prf}}_{\mathrm{BLAKE3}}(\mathcal{B}_{\mathrm{B3}})$ term.

Duplicate, reordered, and timed-out proposals are filtered by the retired-set and deterministic sequence rules without advancing state; dropped packets simply prevent the recovery round-trip from completing, leaving the freshness predicate unmet. □

Remark 4.21 (Defense against Stale-Proposal Replay). Consider an active adversary who intercepts and holds back an update proposal $U = (\mathsf{kem_id}, PQ_{pk}, \sigma_{\mathrm{init}})$ generated by Alice prior to state compromise at t_{comp}, corrupts Alice at t_{comp} to obtain the ephemeral secret key PQ_{sk}, and subsequently delivers U to Bob after t_{rec}. If Bob were to encapsulate to PQ_{pk}, the adversary could decapsulate the response using the leaked PQ_{sk}. Star-Mesh prevents this attack via two complementary defenses: (1) **Proposal timeout and secret zeroization:** The initiator maintains outstanding ephemeral secrets PQ_{sk} in $PQ^{\mathrm{outstanding}}$ only for a bounded timeout duration T_{timeout}. Once T_{timeout} expires without an accepted response, Alice actively zeroizes PQ_{sk} and retires $\mathsf{kem_id}$. A subsequent compromise query cannot recover the zeroized secret. (2) **Predecessor and epoch chaining:** The signature σ_{init} explicitly commits to the sender's current epoch and the transcript hash prev of the last accepted update. If any intermediate communication or ratchet step has occurred, Bob's current prev or expected epoch will not match, causing Bob to reject the delayed proposal. Finally, freshness Condition 5 explicitly requires that the target recovery proposal was generated after t_{rec} using an uncompromised RNG.

5 Security Analysis

5.1 Authentication: Bundle Identity Binding

Security Claim 5.1 (Identity Binding)**.** *Under Assumptions 4.2 (EUF-CMA of ML-DSA-65) and 4.4 (Collision Resistance of SHA3-256), if there exists a QPT adversary $\mathcal{A}_{\mathrm{quant}}$ that forges a valid Pre-Key Bundle with advantage ϵ, then there exist reductions $\mathcal{B}_{\mathrm{DSA}}$ and $\mathcal{B}_{\mathrm{CR}}$ such that*

$$\epsilon \leq \mathbf{Adv}^{\text{euf-cma}}_{\mathsf{ML\text{-}DSA}}(\mathcal{B}_{\mathrm{DSA}}) + \mathbf{Adv}^{\mathrm{CR}}_{H}(\mathcal{B}_{\mathrm{CR}}) + \mathsf{negl}(\lambda).$$

Proof Sketch. Let $\mathcal{B}_{\text{DSA}}$ be a reduction interacting with an EUF-CMA challenger $\mathcal{C}$ for ML-DSA-65. $\mathcal{B}_{\text{DSA}}$ receives a challenge verification key vk^* and embeds it as Bob's identity key $IK_{pk,B}^{DSA}$ in the simulated P2P network. Signing queries from $\mathcal{A}_{\text{quant}}$ are forwarded to $\mathcal{C}$'s signing oracle.

Suppose $\mathcal{A}_{\text{quant}}$ substitutes a pre-key bundle while Bob accepts it as authentic. It must publish a malicious bundle Bundle'_B signed under Bob's identity key $IK_{pk,B}^{DSA}$ that substitutes Bob's pre-keys (SPK_B, PQ_SPK_B, or one-time keys) with keys chosen by the adversary.

The bundle signature covers the full $\mathsf{BundlePayload}$ including the hashes of the OTPK and PQ-OTPK batches (Section 3.3). If the adversary substitutes any pre-key in the batch without altering the batch hash $H(\|_i OTPK_i)$, it outputs a hash collision, breaking collision resistance of SHA3-256 (yielding reduction $\mathcal{B}_{\text{CR}}$). Otherwise, the batch hash or public key changes, so $\mathsf{BundlePayload}$ differs from any payload signed by Bob. Since Bob is honest, this signature must be forged. $\mathcal{B}_{\text{DSA}}$ extracts the message and the signature forgery, submitting it to the EUF-CMA challenger $\mathcal{C}$ to win.

The argument therefore establishes bundle authenticity under unforgeability of ML-DSA-65 and collision resistance of SHA3-256. Any stronger claim about active MITM resistance must separately specify the authentication transcript and cannot reduce a quantum-active adversary's inability to compute an X25519 shared secret to CDH, because X25519 is not quantum-resistant. □

5.2 Post-Quantum Forward Secrecy

Security Claim 5.2 (PQ-FS Bound)**.** *Under Assumptions 4.1, 4.7, and 4.6, subject to stated freshness and deletion conditions, if any QPT adversary $\mathcal{A}_{\text{quant}}$ wins the game $\mathbf{Game}_{\mathcal{A}}^{\text{PQ-FS}}(\lambda)$ (Definition 4.17) at target epoch e with advantage ϵ, then there exist QPT reductions $\mathcal{B}$, $\mathcal{B}_{\text{KDF}}$, and $\mathcal{B}_{\text{B3}}$ such that*

$$\epsilon \leq Q_{\text{prekey}} \cdot \mathbf{Adv}^{\text{ind-cca2}}_{\mathsf{ML\text{-}KEM}}(\mathcal{B}) + \mathbf{Adv}^{\text{hi}}_{\text{KDF}}(\mathcal{B}_{\text{KDF}}) + Q_{\text{sym}} \cdot \mathbf{Adv}^{\text{prf}}_{\text{BLAKE3}}(\mathcal{B}_{\text{B3}}) + \mathsf{negl}(\lambda),$$

where Q_{prekey} is the total number of candidate ML-KEM pre-keys generated across the experiment from which the target bundle is selected, and Q_{sym} bounds the number of symmetric ratchet steps to epoch e.

Proof Sketch. The proof proceeds via a sequence of hybrid games.

Game $\mathbf{H}_0$: The real PQ-FS game $\mathbf{Game}_{\mathcal{A}}^{\text{PQ-FS}}(\lambda)$. $\mathcal{A}_{\text{quant}}$'s advantage in $\mathbf{H}_0$ is ϵ by hypothesis.

Game $\mathbf{H}_1$ (Target Pre-Key Guessing): Identical to $\mathbf{H}_0$, except that at the start of the experiment, the simulator chooses an index $k^* \xleftarrow{\$} \{1, \ldots, Q_{\text{prekey}}\}$ uniformly at random, guessing which of the Q_{prekey} candidate ML-KEM pre-keys generated across all parties will be used in the target session. If $\mathcal{A}_{\text{quant}}$ selects a target session that did not use the k^*-th pre-key, the simulator aborts and outputs a uniform random bit. Because the pre-key index is chosen uniformly at random and independently of the adversary's view, the simulator guesses correctly with probability at least $1/Q_{\text{prekey}}$. Conditioned on guessing correctly, the simulation is syntactically identical to $\mathbf{H}_0$.

Game $\mathbf{H}_2$ (IND-CCA2 Challenge Embedding): Identical to $\mathbf{H}_1$, except that the reduction $\mathcal{B}$ receives an IND-CCA2 challenge (pk^*, ct^*, ss_b) from the ML-KEM challenger. $\mathcal{B}$ embeds pk^* as the guessed k^*-th pre-key $PQ_OTPK_{pk,B}$, sets the corresponding

ciphertext in Alice's M_0 to ct^*, and sets the decapsulated shared secret SS_{PQ2} to ss_b. When $b = 0$, ss_0 is the genuine decapsulation; when $b = 1$, ss_1 is a uniformly random string $u \xleftarrow{\$} \{0,1\}^{32}$ independent of ct^*.

Transition $\mathbf{H}_1 \to \mathbf{H}_2$*:* For all non-target pre-keys, the reduction generates honest keys and decapsulates natively. For the target pre-key k^*, $\mathcal{B}$ uses the IND-CCA2 decapsulation oracle for any ciphertext $ct \neq ct^*$; queries on $ct = ct^*$ are barred by the IND-CCA2 rules, and PQ-FS freshness prevents any $\mathcal{O}^{Reveal}$ on session state that would expose PQ_OTPK_{sk} prior to its erasure. Any distinguishing advantage between the real and random shared secret directly translates to distinguishing $b = 0$ from $b = 1$, yielding:

$$|\Pr[\mathbf{H}_1 \text{ succeeds}] - \Pr[\mathbf{H}_2 \text{ succeeds}]| \leq Q_{\text{prekey}} \cdot \mathbf{Adv}^{\text{ind-cca2}}_{\text{ML-KEM}}(\mathcal{B}).$$

Game $\mathbf{H}_3$: In $\mathbf{H}_2$, $SS_{\text{hybrid}} = \texttt{0xFF} \| SS_{cl} \| SS_{PQ1} \| u$. **Scope note:** this argument covers the case where the PQ-OTPK branch is present (i.e. SS_{PQ2} corresponds to the OTPK encapsulation ct_2). Note that SS_{PQ1} (from the semi-static PQ_SPK encapsulation) remains the real shared secret; only SS_{PQ2} has been replaced by u. In the *SPK-only fallback* (Remark 3.1, when no OTPK is available), the argument instead embeds the challenge in SS_{PQ1}; in that case the security guarantee degrades to session-level PQ-FS (granularity of PQ_SPK rotation), as noted there.

This transition applies Assumption 4.7, **Case 1 (Handshake, Branch 1a)**, with the following identification of components:

- $p = \texttt{0xFF} \,\|\, SS_{cl} \,\|\, SS_{PQ1}$ (known or partially known to $\mathcal{A}_{\text{quant}}$ if DH keys are later revealed).
- $u = SS_{PQ2}$ (computationally uniform by Game $\mathbf{H}_2$).
- $salt = \mathbf{0}^{32}$ (public).
- $info = AD \,\|\, \mathsf{len}_{32}(EK_{pk,A}) \,\|\, EK_{pk,A} \,\|\, \mathsf{len}_{32}(ct_1) \,\|\, ct_1 \,\|\, \mathsf{len}_{32}(ct_2) \,\|\, ct_2$ (public transcript binder, satisfying the ciphertext-binding precondition of Remark 5.3).

Even though p and $salt$ are public, the one-uniform-component argument guarantees that $\mathsf{HMAC}_{\mathbf{0}^{32}}(p \,\|\, u)$ is pseudo-random because u acts as a hidden key for HMAC (dual-PRF property [28]). Concretely, the output is

$$OKM = \mathsf{HKDF}(ikm = SS_{\text{hybrid}},\ salt = \mathbf{0}^{32},\ info,\ \ell = 64),$$
$$\text{where} \quad info = AD \,\|\, \mathsf{len}_{32}(EK_{pk,A}) \,\|\, EK_{pk,A} \,\|\, \mathsf{len}_{32}(ct_1) \,\|\, ct_1 \,\|\, \mathsf{len}_{32}(ct_2) \,\|\, ct_2.$$

By Assumption 4.7 (Case 1, Branch 1a), $OKM = (RK_0, CK_0)$ is computationally indistinguishable from uniform, bounding the transition advantage by: $|\Pr[\mathbf{H}_2] - \Pr[\mathbf{H}_3]| \leq \mathbf{Adv}^{\text{hi}}_{\text{KDF}}(\mathcal{B}_{\text{KDF}})$.

Game $\mathbf{H}_4$: If the target test epoch is $e > 0$, the session advances from initial chain key CK_0 across e symmetric ratchet steps: for each step $k < e$, the simulator derives $MK_k \leftarrow \mathsf{BLAKE3}(CK_k, \texttt{"StarMesh-MK"})$ and advances $CK_{k+1} \leftarrow \mathsf{BLAKE3}(CK_k, \texttt{"StarMesh-CK"})$. By the standard PRF hybrid across at most Q_{sym} steps (Assumption 4.6), each step replaces the derived keys with uniform values, losing at most $Q_{\text{sym}} \cdot \mathbf{Adv}^{\text{prf}}_{\text{BLAKE3}}(\mathcal{B}_{\text{B3}})$. In $\mathbf{H}_4$, the target message key MK_e is completely independent and uniform, yielding $\Pr[\mathcal{A} \text{ wins } \mathbf{H}_4] = 1/2$.

Combining Advantages: Summing the hybrid transitions gives:

$$\begin{aligned}\epsilon &= |\Pr[\mathcal{A} \text{ wins } \mathbf{H}_0] - 1/2| \\ &\leq Q_{\text{prekey}} \cdot \mathbf{Adv}^{\text{ind-cca2}}_{\mathsf{ML\text{-}KEM}}(\mathcal{B}) + \mathbf{Adv}^{\text{hi}}_{\text{KDF}}(\mathcal{B}_{\text{KDF}}) + Q_{\text{sym}} \cdot \mathbf{Adv}^{\text{prf}}_{\text{BLAKE3}}(\mathcal{B}_{\text{B3}}) + \mathsf{negl}(\lambda),\end{aligned}$$

which proves the claimed bound. □

Remark 5.3 (Role of the Classical Component and KEM-Combiner Applicability). The concatenation-combiner theorem of Giacon et al. [19] is proven for two KEMs. Star-Mesh combines X25519 (a Diffie–Hellman function, not natively a KEM) with ML-KEM-768. We apply the theorem with the following protocol-specific adaptation: the X25519 ephemeral key $EK_{pk,A}$ is treated as the KEM's public key, the DH output SS_{cl} as its implicit "shared secret", and the ephemeral key itself is bound into the HKDF $info$ string in place of an explicit ciphertext. Security of the classical component then additionally relies on the DDH / PRF-ODH assumption over Curve25519 (Assumption 4.3). Binding $EK_{pk,A}$, ct_1, and ct_2 into $info$ satisfies the ciphertext-binding precondition of [19], guarding against related-ciphertext and re-encapsulation attacks. This is a protocol-specific adaptation, not a direct application of the theorem; the DDH / PRF-ODH assumption is required in addition to HKDF-PRF.

If X25519 remains computationally secure (no CRQC), SS_{cl} alone makes SS_{hybrid} pseudorandom under the stated assumptions, providing classical forward secrecy. The ML-KEM component provides additional PQ-FS against a CRQC. The two components are *complementary*: security holds if either is secure under the modeled assumptions.

5.3 Post-Compromise Security

Remark 5.4 (PCS healing — relationship to Theorem 4.20). Security Claim 5.2 establishes PQ-FS for the initial handshake. Post-Compromise Security after an authenticated ratchet update is covered by Theorem 4.20, whose bound is:

$$\begin{aligned}\epsilon_{\text{PCS}} \leq\ & Q_{\text{sess}} Q_{\text{upd}} Q_{\text{KEM}}\, \mathbf{Adv}^{\text{ind-cca2}}_{\mathsf{ML\text{-}KEM}}(\mathcal{B}_{\text{KEM}}) + 2 Q_{\text{sess}} Q_{\text{upd}}\, \mathbf{Adv}^{\text{euf-cma}}_{\mathsf{ML\text{-}DSA}}(\mathcal{B}_{\text{DSA}}) \\ & + Q_{\text{KDF}}\, \mathbf{Adv}^{\text{prf}}_{\text{HMAC}}(\mathcal{B}_{\text{KDF}}) + \mathbf{Adv}^{\text{prf}}_{\text{BLAKE3}}(\mathcal{B}_{\text{B3}}) + \mathsf{negl}(\lambda).\end{aligned}$$

The reduction in Theorem 4.20 guesses the target session, update, and KEM branch, paying the joint factor $Q_{\text{sess}} Q_{\text{upd}} Q_{\text{KEM}}$, and separately handles mutual update authenticity (σ_{init} and σ_{resp}) via an EUF-CMA reduction losing $2Q_{\text{sess}} Q_{\text{upd}}$. The KDF hybrid pays $Q_{\text{KDF}} \cdot \mathbf{Adv}^{\text{prf}}_{\text{HMAC}}(\mathcal{B}_{\text{KDF}})$ over sequential $\mathsf{HKDF\text{-}Extract}$ evaluations under fresh salt SS_{PQ}, followed by the BLAKE3 message-key derivation PRF term $\mathbf{Adv}^{\text{prf}}_{\text{BLAKE3}}(\mathcal{B}_{\text{B3}})$. Corollary 5.5 records the qualitative healing statement; quantitative PCS security is entirely captured by Theorem 4.20.

Corollary 5.5 (PCS Healing)**.** *Under the conditions of Theorem 4.20, confidentiality of all epochs e' satisfying conditions* (a), (b), *and* (c) *of Definition 4.18 is restored after one complete PQ ratchet round-trip, under the IND-CCA2 security of ML-KEM-768.*

Proof Sketch. We structure the healing argument and its operational boundaries as follows.

In particular, note the following limitations. First, out-of-order delivery: if the PQ-responder's ciphertext reply for a *kem_id* is delayed, the round-trip may not complete within a bounded window; healing is delayed accordingly. Second, dropped PQ-step messages: if *kem_id* or its matching ciphertext is permanently lost (e.g., via DHT mailbox TTL expiry or eviction rather than mere delay), the round-trip cannot complete. In this

case the PQ-initiator must retire the entry and re-initiate a fresh PQ step with a new keypair; retry policy and timeout values are implementation concerns left to a full protocol specification. Third, skipped-key cache: message keys cached in $\mathcal{K}_{skipped}$ from epochs before t_{rec} are not covered by this corollary (those message keys are already exposed by the prior compromise). Fourth, PQ ratchet counter: if $\mathsf{pq_step_counter} > 0$ at t_{comp}, the PQ-initiator may not send a fresh PQ_{pk}^{loc} until the counter expires (at most Δ additional messages), delaying the start of the round-trip. Fifth, and critically, *identity key survival and signing-oracle isolation*: recovery proceeds in two cases.

- **Passive-after-compromise adversary**: an adversary that reads session state at t_{comp} but thereafter performs no active injections, modifications, or signing-oracle queries. Against this adversary class, healing does not require identity key isolation (IK_{sk}^{DSA} may be exposed). However, recovery is *not unconditional*: it strictly depends on: (i) ML-KEM-768 IND-CCA2 security, (ii) RNG integrity after t_{rec} (Assumption 4.11), and (iii) the deletion schedule (Assumption 4.10). Because the adversary is passive post-compromise, honest endpoints exchange authentic PQ_{pk}^{loc} and ct. By Assumption 4.11, PQ_{sk}^{loc} is sampled with uniform randomness unobserved by the adversary. By ML-KEM-768 IND-CCA2 security, the shared secret SS_{PQ} is computationally indistinguishable from a uniform random string. By Assumption 4.7, HKDF extraction with uniform SS_{PQ} produces an updated entropy pool EP' and new keys $(RK', \vec{CK})$ that are pseudorandom and computationally independent of the compromised pre-recovery root key. By Assumption 4.10, PQ_{sk}^{loc} and pre-recovery keys are zeroized upon update completion. Confidentiality of post-update epochs is thus restored without relying on DSA unforgeability.

- **Active adversary**: one that continues to inject, modify, or replay messages after t_{comp}. Against this stronger model, this corollary requires the full Assumption 4.12: IK_{sk}^{DSA} is not exfiltrated *and* the adversary cannot query the enclave signing oracle $\mathcal{O}^{Sign}$ on the target recovery update transcripts at any time during the experiment (standard EUF-CMA non-triviality). If either condition fails, the adversary can forge or replay σ_{init} and σ_{resp} indefinitely, and healing cannot be guaranteed against this adversary class.

In summary, PCS holds against a passive-after-compromise adversary under ML-KEM IND-CCA2, RNG recovery, and erasure enforcement, and additionally requires Assumption 4.12 against an active adversary. □

6 Metadata Resistance

Star-Mesh achieves metadata resistance through two complementary mechanisms.

DHT Mailbox Indirection. Each node maintains a *mailbox* in the Kademlia DHT keyed by a pseudonymous address $\mathsf{addr}_i = H(IK_{pk,i}^{DSA} \| \mathsf{epoch})$, rotated periodically. Senders deposit encrypted messages at the mailbox address; recipients perform constant-rate polling. This severs the direct IP-to-recipient link exploitable by a passive observer of DHT traffic.

Onion-Routing Overlay. Message delivery and mailbox retrieval are routed through a libp2p-based onion-routing overlay following the Loopix [15] architecture. Each message is layered with Sphinx-format encryption [16] over a randomly sampled sequence of mix

nodes. Dummy cover traffic is generated at a constant rate λ_{cover} to prevent traffic analysis via timing or volume.

Remark 6.1 (Sender-Receiver Unlinkability — Heuristic Modeling). Under Assumption 4.9 (at most $f < 1/2$ of mix nodes are adversarially controlled) and assuming constant-rate cover traffic, the probability that $\mathcal{A}_{\text{quant}}$ links a sending event to a receiving event is modeled heuristically as at most $\frac{f}{1-f} \cdot \frac{1}{n_{\text{mix}}} + \mathsf{negl}(\lambda)$, where n_{mix} is the number of honest mix nodes on the path. This expression is a conceptual approximation derived from the corruption-fraction threat framework rather than a formal cryptographic bound; see the note below.

The formulation above is an *informal modeling heuristic*, adapted from the corruption-fraction threat framework underlying Loopix [15]. We do not claim that Section 4.3 of [15] states this closed-form expression; that work evaluates empirical end-to-end anonymity under Poisson mixing and constant-rate cover traffic. Our use of $\frac{f}{1-f} \cdot \frac{1}{n_{\text{mix}}}$ serves strictly to parameterise system-level risk in Star-Mesh under Assumption 4.9. We make no new claims about mixnet anonymity beyond this heuristic model. Star-Mesh's primary contribution is the integration of this routing overlay with the hybrid post-quantum ratchet layer.

Empirical Queue Simulation Results. To evaluate the packetization discipline under the signed ratchet design required by Theorem 4.20, the repository provides a reproducible discrete-time queue simulation in `poc/python/mixnet_sim.py`. The experiment simulates 30 mix nodes over three mix hops, with node service capacity $C = 2$ packets/slot, symmetric cover traffic ($\lambda_{\text{cover}} = \lambda$), and an adversarial compromise fraction of 20% ($f = 0.2$). Because state compromise cannot be detected locally, *every* periodic KEM ratchet update (triggered every $\Delta = 50$ messages) carries mutual ML-DSA-65 identity signatures (*UpdateAuth*, 3,309 bytes each) alongside ML-KEM-768 keys (1,184 B) or ciphertexts (1,088 B). Bundled with standard 192-byte application payloads, these signed update messages total 4,685 B (initiator) and 4,589 B (responder). To preserve the constant-packet-size discipline of the Sphinx mixnet (1,500-byte MTU with 96-byte onion routing headers), the transport layer fragments each signed update into a bounded train of 4 Sphinx packets ($\lceil 4{,}685/1{,}404 \rceil = 4$), while intermediate conversational messages (98% of transmissions) traverse the network as single 1,500-byte Sphinx packets without fragmentation. In the simulation, an authenticated update is counted as delivered if and only if all 4 fragments successfully traverse the mixnet to the destination.

We formally define the *Entry–Exit Linkability* metric as the empirical fraction of delivered messages for which an adversary concurrently controls both the ingress mix node (first hop) and egress mix node (last hop) along the packet's onion-routed path:

$$\text{Linkability} \coloneqq \frac{1}{|\mathcal{M}_{\text{del}}|} \sum_{m \in \mathcal{M}_{\text{del}}} \mathbb{I}(\text{entry}(m) \in \mathcal{C} \ \wedge \ \text{exit}(m) \in \mathcal{C}),$$

where $\mathcal{C}$ denotes the set of compromised mix nodes ($|\mathcal{C}|/n_{\text{total}} = f$) and $\mathcal{M}_{\text{del}}$ is the set of completely delivered messages. This metric serves as an empirical lower-bound proxy for path-correlation vulnerability under fractional node compromise. Table 1 reports the empirical queueing dynamics across 13 simulation runs spanning offered application loads $\lambda \in \{1.0, 5.0, 10.0, 20.0\}$ and four node churn rates under this signed protocol discipline.

The simulation findings validate the architectural hypothesis:

1. **Transport MTU Compliance and the Burst-Fingerprint Limitation:** Across all simulated packets in the 13 simulation runs of Table 1, the Sphinx packet MTU

Table 1: Empirical mixnet queue simulation results for the signed ratchet design across 13 simulation runs (30 nodes, 3 hops, capacity $C = 2$ pkts/node/slot, 1,500-byte MTU, 4 fragments per signed update, from `poc/python/mixnet_sim.py`).

Load (λ) (pkts/slot)	**Churn Rate** (fraction/slot)	**Packet MTU Violations**	**Delivery Rate**	**p95 Delay** (slots)	**Entry–Exit Linkability**
1.0	0.0	**0**	100.0%	3.0	12.8%
5.0	0.0	**0**	100.0%	4.0	3.5%
10.0	0.0	**0**	95.1%	279.0	2.2%
20.0	0.0	**0**	31.3%	3,258.0	4.7%
1.0	0.1	**0**	65.3%	3.0	5.4%
5.0	0.1	**0**	62.8%	4.0	0.8%
10.0	0.1	**0**	52.1%	7.0	3.6%
1.0	0.2	**0**	39.7%	3.0	2.1%
5.0	0.2	**0**	38.3%	3.0	4.6%
10.0	0.2	**0**	31.8%	5.0	2.1%
1.0	0.3	**0**	23.9%	3.0	0.7%
5.0	0.3	**0**	22.8%	3.0	2.4%
10.0	0.3	**0**	19.1%	4.0	2.3%

violation rate was strictly **0%**. While raw signed updates (> 4.5 KB) cannot fit within a single Ethernet MTU, fragmenting them into 4 standard 1,500-byte Sphinx packets satisfies the network MTU constraint. However, this multi-packet fragmentation introduces an operational limitation: the periodic transmission of a 4-packet burst every $\Delta = 50$ messages creates an identifiable traffic volume burst fingerprint. This burst pattern contradicts theoretical claims of metadata security without leakage: a passive network adversary observing flow volume or packet inter-arrival times across mix nodes can correlate these periodic 4-packet bursts to identify communicating pairs, even though individual packets are encrypted and padded to uniform size. Routine conversational messages traverse the network as single packets (98% of transmissions), accentuating the contrast with ratchet update bursts.

Mitigating Burst Fingerprints. To eliminate this traffic-analysis vector in production deployments, several countermeasures may be deployed: (i) *dummy burst injection*, wherein nodes periodically inject dummy 4-packet trains at randomized (e.g., Poisson-distributed) intervals independent of ratchet updates; (ii) *uniform train padding*, wherein routine conversational messages are padded with dummy Sphinx packets to constant multi-packet trains ($k = 4$); or (iii) *dribble fragmentation*, wherein KEM public keys, ciphertexts, and signatures are streamed incrementally across consecutive single-packet conversational messages (similar to the chunked streaming in SPQR [4]), trading a wider post-compromise exposure window for a smooth, single-packet traffic profile.

2. **Impact of Multi-Packet Fragment Trains on Queueing and Churn:** Under zero churn, delivery remains near-lossless ($\geq 99.9\%$) for loads up to $\lambda = 5.0$. The 4-packet fragment trains injected during KEM updates slightly increase queue pressure at the saturation threshold ($\lambda = 10.0$, where p95 delay reaches 279 slots). Under churn (10%–30%), the delivery rate for signed updates is bounded by the probability that all 4 fragments survive intermediate node dropouts, reinforcing

the need for transport-layer retransmissions and acknowledgments. This collapse is an analytical artifact of queueing capacity saturation in the discrete-event model rather than a protocol or cryptographic bottleneck: because cover traffic is generated at $\lambda_{\text{cover}} = \lambda$, total network ingress is 2λ packets/slot. Traversing 3 hops requires $3 \times 2\lambda = 6\lambda$ per-hop forwarding operations per slot. With 30 nodes each serving $C = 2$ packets/slot, aggregate network capacity is strictly 60 packet-operations/slot. At $\lambda = 10$, utilization reaches 100% ($\rho = 1.0$); at $\lambda = 20$, offered demand is 120 packet-operations/slot (200% overload), causing queues to grow monotonically throughout the 5,000-slot run and unserved packets remaining in queues at run termination to be counted as undelivered. Furthermore, the simulation operates in abstract discrete slots without an explicit physical time mapping; in operational mixnet deployments, mix nodes process thousands of packets per second, and per-hop Poisson delays are calibrated to physical milliseconds. Under churn (10%–30%), delivery rate degrades proportionally to route interruption, highlighting that transport-layer reliability requires end-to-end ACKs and retransmissions.

3. **Scope Limitation:** We emphasize that this queue model simulates steady-state ratchet exchanges; one-time handshake messages ($M_0 = 7{,}565$ B; 6,477 B for SPK-only) and signed pre-key bundles require multi-packet Sphinx trains (6 fragments for M_0 as $\lceil 7{,}565/1{,}404 \rceil = 6$, or 5 fragments for SPK-only fallback as $\lceil 6{,}477/1{,}404 \rceil = 5$) or direct mailbox deposition.

7 Bandwidth and Performance Analysis

A critical challenge in adopting PQC over P2P networks is the substantially increased size of ciphertexts and keys relative to classical primitives. Table 2 summarises the relevant ML-KEM-768 and X25519 sizes.

Table 2: Primitive sizes relevant to Star-Mesh.

Primitive	**Public Key**	**Ciphertext / DH Output**	**Shared Secret**
X25519	32 B	32 B	32 B
ML-KEM-768	1184 B	1088 B	32 B
ML-DSA-65	1952 B	N/A (sig: 3,309 B)	N/A

Table 3 compares the per-connection overhead of a classical Signal-style protocol against Star-Mesh.

Amortised Cryptographic Cost. The cryptographic overhead of the post-quantum ratchet is governed by the requirement of post-compromise security (Theorem 4.20):

1. **Primary Specification: Signed Ratchet for PCS (Theorem 4.20):** Because state compromise is silent and undetectable, an endpoint cannot determine when its ratchet state has been exfiltrated. To guarantee healing against an adversary in possession of the compromised symmetric ratchet state, *every* periodic KEM update must authenticate both the initiator's public key proposal and the responder's ciphertext response under their long-term identity keys via ML-DSA-65 signatures (*UpdateAuth*, 3,309 bytes each). This requires $1{,}184 + 3{,}309 = 4{,}493$ bytes for the initiator step and $1{,}088 + 3{,}309 = 4{,}397$ bytes for the responder reply, totaling 8,890 bytes per update round-trip. Amortised across both conversation directions

Table 3: Bandwidth overhead comparison. Classical Signal handshake size is derived from the X3DH specification [8]: in X3DH, Alice fetches Bob's pre-key bundle from the distribution server, and her initial transmission to Bob carries her identity key IK_A (32 B), ephemeral key EK_A (32 B), one-time pre-key identifiers ($\approx$ 8 B), and protocol framing, totalling approximately 150 bytes with an initial encrypted message (118 bytes without an OTPK). Bob does not receive his own SPK or OTPK back from Alice. Star-Mesh handshake figures follow the M_0 definition of Section 3.4: $IK^{DSA}_{pk,A}$ (1,952 B) + $IK^{DH}_{pk,A}$ (32 B) + $EK_{pk,A}$ (32 B) + ct_1 (1,088 B) + optionally ct_2 (1,088 B) + 32 B confirmation tag + σ_0 signature (3,309 B) + 32 B framing/identifiers = 7,565 B with OTPK (7,533 B raw cryptographic payload), 6,477 B without (6,445 B raw cryptographic payload). With a 1,404-byte Sphinx payload capacity, transmitting M_0 requires a 6-packet Sphinx train ($\lceil 7{,}565/1{,}404 \rceil = 6$, and 5 packets for SPK-only: $\lceil 6{,}477/1{,}404 \rceil = 5$). The 1,952-byte $IK^{DSA}_{pk,A}$ is transmitted on *every* handshake as specified; implementations that cache a correspondent's identity key after first contact can amortise this cost across a relationship rather than a session.

Component	**Classical (Signal)**	**Star-Mesh**	**Overhead**
Star-Mesh handshake rows include the 32-byte tag_A confirmation tag.			
Handshake, with OTPK (bytes)	$\approx$ 150 [8]	$\approx$ 7,565	+7,415
Handshake, SPK-only fallback	$\approx$ 118 [8]	$\approx$ 6,477	+6,359
Ratchet header (non-PQ step)	32 (X25519 pk)	32 (X25519 pk)	0
Ratchet step, unsigned (raw KEM pk)	N/A	+1,184 (KEM pk)	+1,184
Ratchet reply, unsigned (raw KEM ct)	N/A	+1,088 (KEM ct)	+1,088
Ratchet step, signed ($pk + \sigma_{\mathsf{ML\text{-}DSA}}$)	N/A	+4,493	+4,493
Ratchet reply, signed ($ct + \sigma_{\mathsf{ML\text{-}DSA}}$)	N/A	+4,397	+4,397

over ratchet interval $\Delta = 50$ (100 messages total in the alternating single-initiator baseline), the per-message cryptographic overhead is:

$$\frac{(|PQ_pk| + |\sigma_{\mathsf{ML\text{-}DSA}}|) + (|PQ_ct| + |\sigma_{\mathsf{ML\text{-}DSA}}|)}{2\Delta} = \frac{8{,}890}{100} = 88.90 \text{ bytes/message.}$$

Because each signed update payload exceeds the 1,500-byte MTU, each update is fragmented by the transport layer into a bounded train of 4 Sphinx packets. In the worst-case scenario where both communicating endpoints trigger concurrent updates at the expiration of their independent counters (Remark 3.2), each party conducts an independent signed update round-trip, injecting $2 \times 8{,}890 = 17{,}780$ bytes over the 100 conversational messages, yielding an upper bound of 177.80 bytes/message.

2. **Theoretical Lower Bound (Unsigned Baseline):** If an adversary were assumed never to compromise the internal symmetric state (or if compromise were signaled out-of-band), updates could theoretically omit signatures and rely solely on symmetric chain keys. In that unauthenticated baseline, the raw ML-KEM-768 material would add 1,184 + 1,088 = 2,272 bytes per round-trip, or 22.72 B/msg (45.44 B/msg in concurrent dual-initiation). However, as proven in Section 4.5, this unsigned baseline *fails to provide post-compromise security* under state compromise. Star-Mesh therefore specifies the signed design (88.90 B/msg baseline, 177.80 B/msg worst case) as its standard protocol.

Decoupled Overlay and DHT Transport Overhead. We emphasize an important architectural separation: the cryptographic ratchet wire overhead above is strictly decou-

pled from the transport and routing overhead of the underlying decentralized network. A peer-to-peer messaging system over a Kademlia DHT incurs operational traffic for key lookup, record maintenance, and mailbox polling; however, this traffic is a property of the P2P routing overlay and is identical whether sessions use classical or post-quantum ratcheting.

Specifically, at the data layer, a mailbox routing record introduces a minimal static addressing footprint (a 20-byte node ID plus an 8-byte TTL field, and a 32-byte polling request header). However, network-layer transport costs cannot be represented as a static per-message addend:

1. **Iterative DHT Lookups:** Resolving a recipient's pre-key bundle or mailbox address requires iterative Kademlia queries ($\alpha = 3$ concurrent paths over $O(\log N)$ hops), where each response exchanges lists of up to $k = 20$ contact tuples (26–38 bytes each). While lookups are amortised across ongoing sessions via routing-table and contact caching, initial discovery or route repairs under churn generate kilobyte-scale signaling overhead.

2. **Replication and Churn:** Maintaining mailbox availability requires periodic republishing to the k closest nodes; in networks with high churn, maintenance bandwidth scales with churn rate.

3. **Continuous Polling vs. Message Arrival Rate:** Metadata-resistant constant-rate polling consumes network bandwidth as a function of time (bytes per second), not per application message. Under low message arrival rates, polling traffic amortised per delivered message is naturally substantially higher than under continuous dialogue.

4. **Mixnet Packet Padding and Fragmentation:** As detailed in Section 6, messages and polling queries traversing the mixnet overlay are encapsulated in constant-size Sphinx packets padded to a 1,500-byte MTU. Within this packet discipline, the 88.90-byte amortised PQ ratchet overhead is absorbed across periodic 4-packet Sphinx fragment trains every Δ messages, preserving the constant-packet-size discipline across the network.

Consequently, a complete empirical accounting of network-level bandwidth under realistic churn and polling models requires high-fidelity network emulation (planned in our engineering roadmap), whereas the 88.90 bytes/message reported here represents the invariant cryptographic overhead mandated to achieve post-compromise security under state compromise.

Computational Cost. ML-KEM-768 encapsulation and decapsulation execute in well under 1 ms on modern hardware [1], triggered at most once every 50 messages. To validate the computational practicality of our protocol, we micro-benchmarked our Rust prototype over 1,000 iterations on an Apple M1 Air (8-core CPU, macOS, `rustc` 1.97.1, using RustCrypto `ml-kem` v0.3 and `ml-dsa` v0.1.1), compiled in Rust `release` mode (`opt-level = 3`, link-time optimisation enabled, `cargo build --release`). The micro-benchmark is executed in release mode and can be reproduced with `make rust-poc` (or directly via `cargo run --release --manifest-path poc/rust/Cargo.toml`). The benchmark is implemented as a hand-rolled 1,000-iteration timing loop using `std::time::Instant` (`poc/rust/src/main.rs`, Phase 5), reporting mean, standard deviation, and percentiles computed from the sorted sample. This methodology is representative but not equivalent to a statistically rigorous harness such as `criterion`: there is no warm-up phase, no outlier

detection, and statistics are computed over a single run. The figures should therefore be read as indicative performance magnitudes rather than confidence-bounded latency guarantees. Under this platform:

- **PQ-X3DH Handshake** (initiator + responder): mean execution time of 1,530.23 μs ($\approx$ 1.53 ms; std dev: 487.57 μs; p50: 1,367.29 μs; p95: 2,552.88 μs; p99: 3,430.04 μs).
- **PQ Ratchet Round-trip** (init + encaps + decaps with mutual ML-DSA-65 signing and verification): mean execution time of 1,530.92 μs ($\approx$ 1.53 ms; std dev: 669.22 μs; p50: 1,381.96 μs; p95: 2,847.63 μs; p99: 3,712.83 μs).

These low-latency execution times confirm the computational practicality of the scheme for consumer endpoints. The handshake latency includes ML-DSA-65 signing (σ_0) by Alice and signature verification by Bob, four ML-KEM-768 operations (two encapsulations by Alice and two decapsulations by Bob for ct_1 and ct_2 in the full handshake with OTPK), four X25519 scalar multiplications, and KDF/hash logic. The ratchet round-trip includes one ML-KEM-768 key generation, encapsulation, and decapsulation, HKDF root mixing, and mutual ML-DSA-65 signature generation and verification (σ_{init} and σ_{resp}) mandated by Theorem 4.20. Even with full mutual asymmetric signing on every periodic update, the entire round-trip completes in approximately 1.5 ms, which is imperceptible to users; for reference ML-KEM-768 reference implementation performance figures, see the NIST FIPS 203 supporting documentation [1]. The dominant computational overhead in StarMesh is expected to be the mixnet onion routing (Sphinx packet construction [16]), which requires multiple asymmetric operations per hop and is independent of the PQC primitives.

8 Assumptions and Limitations

We consolidate the protocol's assumptions and known limitations here for clarity.

1. **ML-DSA-65 EUF-CMA security** (Assumption 4.2): ML-DSA-65 is secure against existential forgery under chosen-message attacks.
2. **DDH / PRF-ODH on Curve25519** (Assumption 4.3): DDH and PRF-ODH on Curve25519 are hard against classical PPT adversaries prior to CRQC, required for ephemeral DH pseudorandomness and implicit authentication (DH_1).
3. **Module-LWE hardness** (Assumption 4.1): ML-KEM-768 is IND-CCA2 secure under MLWE with $n = 256$, $k = 3$.
4. **HKDF as PRF and Extractor** (Assumption 4.5): HKDF-SHA3-256 is modeled as a two-stage extract-then-expand PRF and computational extractor following [28].
5. **KDF Security under Secret and Public Salt** (Assumption 4.7): In the post-quantum ratchet, extraction operates under secret salt SS_{PQ} as a standard PRF; in the handshake, extraction operates under public salt via dual-PRF extraction whenever the concatenated IKM contains at least one uniform component.
6. **BLAKE3-KDF as PRF** (Assumption 4.6): BLAKE3's keyed hashing mode is modeled as a PRF; required for pseudorandomness of per-message keys MK_i and hence for message confidentiality.
7. **Deletion schedule enforced** (Assumption 4.10): An operational assumption; not proven by the protocol logic but required for FS and PCS.

8. **RNG integrity after t_{rec}** (Assumption 4.11): The RNG is clean from t_{rec} onward. A compromised RNG during PQ ratchet key generation makes PCS unachievable for that update.

9. **Identity key isolation across state compromise** (Assumption 4.12): *Against active adversaries, PCS holds only if the identity signing key (IK_{sk}^{DSA}) remains uncompromised during and after ephemeral state compromise.* On an unprotected device, total device expropriation leaking both session memory and the identity signing key allows an active adversary to forge update signatures, rendering recovery unachievable against active tampering. This assumption requires the identity key to reside in a hardware-isolated environment (HSM, secure enclave, or protected keystore) inaccessible to the Corrupt oracle. (In contrast, against a passive-after-compromise adversary who only eavesdrops post-compromise, recovery does not depend on identity key isolation, but still fundamentally depends on ML-KEM-768 IND-CCA2 security, post-t_{rec} RNG integrity, and deletion schedule enforcement; it is not "unconditional".) The ratchet's PCS addresses a distinct threat from what the identity key alone provides: static identity keys supply authentication, whereas the ratchet continuously refreshes quantum-resistant symmetric confidentiality, using the hardware-isolated signing key strictly to prevent active ciphertext substitution during healing. A reviewer asking "why does the ratchet add anything beyond the identity key?" should read Section 4: the EUF-CMA term in Theorem 4.20 is a guard against active substitution, not the primary confidentiality mechanism; ML-KEM IND-CCA2 provides the confidentiality recovery.

10. **Mixnet honesty fraction** (Assumption 4.9): At most $f < 1/2$ of mix nodes are adversarially controlled.

11. **DHT Sybil resistance** (Assumption 4.8): The DHT is operationally protected against Sybil and eclipse attacks; this is not proven by the protocol.

12. **Out-of-order delivery and PCS** (Corollary 5.5): PCS restoration is conditional on the round-trip completing; delayed network delivery defers healing.

13. **Dropped PQ-step messages**: If an authenticated *kem_id* entry or its matching ciphertext is permanently lost (rather than merely delayed), the PQ-initiator must timeout and re-initiate with a fresh keypair. Retry policy is an implementation concern.

14. **Skipped-key cache and PCS**: Keys cached in $\mathcal{K}_{skipped}$ before t_{rec} remain compromised even after the PQ ratchet round-trip.

15. **Confirmation tags and half-open sessions**: tag_A gives Bob one-sided explicit key confirmation in M_0; the session is not mutually confirmed until Alice receives and verifies Bob's authenticated response $tag_B = \mathsf{HMAC}_{\text{SHA3-256}}(K_{conf}, \texttt{"Bob"} \| M_0^{core})$. Until that response is received, Alice must treat the session as pending one-sided confirmation only and must not send application data that requires mutual confirmation guarantees. A missing or unverifiable tag_B is an authentication/availability failure, not implicit permission to proceed.

16. **Deniability loss from transcript signatures:** Every PQ ratchet update carries a mutual ML-DSA-65 signature (σ_{init}, σ_{resp}) bound to both parties' long-term identity keys. These signatures are publicly verifiable: any third party given the transcript and

the identity public keys can cryptographically confirm that a conversation took place between A and B. This eliminates the deniability property present in the classical Double Ratchet, where ECDH key exchange provides implicit authentication but leaves no verifiable transcript evidence. The design choice is deliberate: mutual ML-DSA authentication is required for PCS against an adversary who has compromised symmetric ratchet state and could otherwise inject adversarial ciphertexts during recovery (Theorem 4.20). Operators requiring deniability should consider alternative authentication mechanisms: ring-signature or group-signature schemes over identity keys (providing authentication without individual binding), MAC-based mutual authentication using a pre-established shared secret (as in the classical Double Ratchet), or a two-phase design where deniable implicit authentication covers normal messages and non-deniable explicit signatures are used only on signed ratchet updates, with the signed transcript treated as a recoverable attestation log rather than repudiable chat history. Formal deniability in post-quantum asynchronous ratchets remains an open direction; the tensions between post-quantum KEM authentication and deniability in Signal-like handshakes are explored in [37].

9 Related Work

Signal (PQXDH, SPQR) and Apple PQ3. Signal's PQXDH [3] introduced post-quantum key encapsulation into the initial handshake to provide PQ-FS against HNDL attacks. Both PQXDH and Star-Mesh's PQ-X3DH achieve asynchronous handshake completion by fetching pre-keys ahead of dialogue; what distinguishes Star-Mesh is that pre-keys are distributed over a decentralized, serverless Kademlia DHT rather than a trusted central server. Under Apple's four-tier taxonomy of messaging security (Levels 0–3) [7], initial-handshake-only PQC represents Level 2. Reaching Level 3 requires post-quantum ratcheting throughout ongoing sessions. Recently, Signal introduced the Sparse Post-Quantum Ratchet (SPQR / Triple Ratchet) [4–6], developed jointly by Signal engineers and researchers from PQShield and academia. SPQR integrates ML-KEM-768 into the Double Ratchet by streaming erasure-coded chunks of large KEM encapsulation keys (1,184 B) and ciphertexts (1,088 B) across successive conversational messages (sliced into small chunks, on the order of tens of bytes per message depending on erasure-coding parameters and transport framing) and overlapping ciphertext generation via the "ML-KEM Braid" to continuously derive fresh 32-byte post-quantum shared secrets. Similarly, Apple's iMessage PQ3 [7] introduced periodic in-band ML-KEM re-keying (performing a post-quantum ratchet approximately every 50 messages) into a centralized messaging service.

In contrast to the centralized transport assumed by SPQR and PQ3, Star-Mesh targets a decentralized DHT and Sphinx mixnet setting. In Star-Mesh, post-quantum authentication is enforced across all critical phases: Bob's pre-key bundle is authenticated via his long-term ML-DSA-65 identity key (preventing DHT pre-key substitution), the initial handshake message M_0 carries Alice's ML-DSA-65 signature σ_0 over her classical and KEM keys (providing explicit post-quantum initiator authentication and preventing active CRQC impersonation or key-substitution attacks during session establishment), and post-compromise ratchet updates mandate mutual ML-DSA-65 signatures (σ_{init} and σ_{resp}), which prevent an adversary holding compromised symmetric ratchet state from injecting adversarial ciphertexts during recovery.

Session Protocol and Briar. Session [23] routes messages through an onion network to achieve metadata resistance and eliminates central servers. Briar [24] routes through Tor. Both achieve excellent decentralization and metadata resistance but use exclusively classical X25519 for confidentiality, leaving them vulnerable to HNDL attacks. Star-Mesh integrates hybrid PQC into a similar decentralized architecture. The technical obstacle these systems do not solve is the size incompatibility between post-quantum primitives and onion-routing packet disciplines: Tor and Sphinx-based mixnets enforce fixed-size cells or constant-size packets to prevent traffic analysis, but ML-DSA signatures and ML-KEM ciphertexts exceed these bounds, requiring non-trivial fragmentation or padding strategies that directly weaken the traffic-analysis resistance those networks are designed to provide.

Double Ratchet Formal Analysis. Alwen, Coretti, and Dodis [10] formalized the Double Ratchet by modularizing it into Continuous Key Agreement (CKA) and forward-secure AEAD, generalizing the ratchet to generic KEM-based constructions—the closest theoretical foundation for our asynchronous ML-KEM ratchet. Cohn-Gordon et al. [11] provided the first multi-stage formal security analysis of the Signal Double Ratchet. Cremers and Zhao [12] extended this to a model capturing strong compromise resilience, temporal privacy, and immediate decryption simultaneously. Jaeger and Stepanovs [34] derived optimal security bounds for ratcheted key exchange, establishing tight separations between forward secrecy, post-compromise security, and deniability that inform the trade-offs Star-Mesh makes. Poettering and Rösler [35] formalized asynchronous ratcheted key exchange (ARKE) and showed that post-compromise recovery requires a minimum of one round-trip under asynchronous delivery—consistent with our Corollary 5.5. Dowling, Fischlin, Günther, and Stebila [36] analyzed multi-stage key exchange security for the TLS handshake; their stage-based freshness predicates influenced the oracle naming in Section 4. Our security model and argument structure are closely adapted from [11, 12], with aligned freshness/oracle semantics but protocol-specific reformulation for the hybrid post-quantum setting with ML-KEM.

Post-quantum ratchet recovery after a bounded number of epochs has been studied by Auerbach et al. [6] in the context of SPQR; Star-Mesh's explicit contribution is the combination of (i) a formal pre-recovery exposure window under RNG-recovery, deletion, and asynchronous delivery assumptions, (ii) decentralized pre-key distribution over a Kademlia DHT without a trusted server, and (iii) the mixnet-level burst fingerprint analysis and mitigations in Section 6. The technical obstacle these analyses leave open is the adaptation of ratchet freshness semantics to KEM-based key exchange: classical DH-ratchet models assume commutativity and ephemerality properties that do not hold for KEMs, where the encapsulator and decapsulator roles are asymmetric and static public keys must be pre-distributed, forcing a new formal treatment of RNG-compromise recovery and one-time pre-key exhaustion that is absent from existing Double Ratchet proofs.

PQ-MLS and Group Messaging. RFC 9420 [25] defines the base Messaging Layer Security (MLS) protocol for centralized group messaging; post-quantum extensions to MLS are an active area of standardization effort. Star-Mesh targets asynchronous pairwise messaging over a DHT. The two protocol classes are complementary and address different deployment scenarios. The technical obstacle that prevents MLS-style designs from operating over a decentralized DHT is their dependence on a central Delivery Service to impose a total ordering on group operations and maintain tree consistency: without such a server, concurrent pre-key updates produce conflicting tree states that cannot be resolved without a global coordination mechanism, a problem that does not arise in Star-Mesh's

pairwise ratchet setting. A plausible future direction is D-MLS: use a CRDT operation set for signed proposals and commits, retain the MLS parent tree hash and confirmation-tag checks, and apply a deterministic winner rule to concurrent valid commits. This would provide convergence for clients with the same operation set, but would not itself provide Byzantine consensus or prevent DHT eclipse and fork attacks; the full design and required security tests are left to a separate research track in `docs/d-mls-design.md`.

Hybrid KEM Composition. Giacon et al. [19] proved that the concatenation combiner for KEMs achieves IND-CCA2 security if either component KEM is IND-CCA2 secure, provided the KEM ciphertexts are bound into the KDF input and the combiner uses a suitable KDF modeled as a PRF (as established in [19]). Our hybrid KEM construction satisfies this precondition by binding the ciphertexts ct_1, ct_2 and the ephemeral public key $EK_{pk,A}$ into the HKDF $info$ parameter, applying the composition results of [19] with HKDF as the PRF (Assumption 4.5). However, these static composition results do not extend to the stateful ratcheting setting: they bound the advantage of a single IND-CCA2 distinguisher against a one-shot encapsulation, but do not address the session-level forward secrecy and post-compromise recovery properties that arise when KEM shared secrets are fed into a continuously updated ratchet state machine where cached skip-keys and out-of-order delivery create additional exposure windows.

Metadata Resistance. Chaum [14] introduced mix networks; Piotrowska et al. [15] developed the Loopix architecture providing provable anonymity under formal assumptions. The Sphinx packet format [16] provides onion encryption with constant-size packets. Star-Mesh inherits these results directly for its routing layer. The technical obstacle these systems leave unaddressed is the integration of large post-quantum key material into a constant-packet-size discipline: Loopix's anonymity proof assumes all packets are padded to a fixed size, but ML-DSA-87 signatures (> 4.5 KB) and ML-KEM-1024 ciphertexts (1.5 KB) far exceed standard Sphinx MTUs, and naïvely introducing these payloads forces perpetual fragmentation or re-padding that invalidates the constant-size assumption on which the anonymity bound depends. Star-Mesh bounds this by selecting Category 3 parameters (ML-KEM-768, ML-DSA-65), which allow intermediate conversational messages to fit within standard 1,500-byte MTU bounds without fragmentation, confining multi-packet Sphinx trains to periodic authenticated KEM updates (4 fragments every Δ messages) and the initial session handshake (M_0), as detailed in Section 2.5.

10 Conclusion

We introduced a windowed view of recovery in an asynchronous hybrid KEM ratchet. The central artifact is a state machine for authenticated ML-KEM exchanges under delay, loss, duplication, replay, and out-of-order delivery, together with a freshness game that makes RNG recovery and secret deletion explicit. This view turns recovery into a measurable transition: after compromise, a cleanly generated ML-KEM keypair and a completed authenticated round-trip yield fresh post-recovery message keys, while the messages sent before completion occupy an exposure window governed by Δ messages of counter delay plus network round-trip latency (unbounded if the peer is offline). The parameter Δ exposes the design trade-off directly: smaller intervals shorten pre-recovery exposure, while larger intervals amortise KEM traffic.

The resulting theorem has explicit delivery and implementation assumptions and targets the recovery transition itself. ML-DSA authenticates the signed bundle and every periodic

ratchet update; X25519 remains a classical hybrid component, while ML-KEM supplies the recovery contribution against a quantum-capable adversary. The Rust prototype exercises the concurrent implementation state machine and the ProVerif models check selected symbolic invariants. The cryptographic wire-cost of the post-quantum ratchet is 88.90 amortised bytes per message at $\Delta = 50$, which fully accounts for the mutual ML-DSA-65 identity signatures on every update required to guarantee post-compromise security under state compromise (excluding DHT overlay maintenance and mixnet cover traffic).

Key assumptions underlying the pairwise recovery bound are: Module-LWE hardness (Assumption 4.1), ML-DSA unforgeability (Assumption 4.2), the deletion schedule being enforced by the implementation (Assumption 4.10), a clean RNG from t_{rec} onward (Assumption 4.11), and the dual-PRF / one-uniform-component extractor KDF security (Assumption 4.7). The DHT availability and mixnet assumptions are relevant only to deployment-context discussions and do not enter the pairwise confidentiality theorem.

Research Positioning. The contribution is a protocol-design and security-analysis framework for reasoning about asynchronous recovery as a timed, stateful transition. It provides a basis for implementation and deployment work; production use would additionally require implementation hardening, empirical evaluation, interoperability testing, formal verification, and operational abuse-resistance analysis.

Future Work. Several directions remain open. First, a full composable security proof (e.g., in the UC framework [31]) for the complete protocol would be desirable; the present ProVerif models provide symbolic evidence but not a computational composable treatment. Second, group messaging extensions analogous to PQ-MLS [25] should be explored for the DHT setting. Third, an implementation and empirical performance evaluation on resource-constrained devices (IoT, mobile) would validate the practical viability of the design. Fourth, the confirmation-tag mechanism for M_0 requires a full mutual-confirmation response path and computational analysis under replay, timeout, and transcript-rebinding attacks.

Acknowledgements

AI-assisted writing tools were used solely for language refinement and typographical editing of this manuscript. All protocol designs, security definitions, security arguments, and technical contributions are entirely the work of the author.

A Complete Protocol State Machine and Message Framing

This appendix provides the full, normative specification of the Star-Mesh state machine, message framing, asynchronous concurrent KEM buffer, and staged commit rules.

A.1 Endpoint State Representation

Each communicating peer maintains a persistent state record containing the following fields:

- **Session context:** Session ID $\mathsf{sid} \in \{0,1\}^{32}$ and protocol role $\mathsf{role} \in \{\text{init}, \text{resp}\}$.
- **Identity keys:** ML-DSA-65 keypair $(IK_{sk}^{DSA}, IK_{pk}^{DSA})$ (hardware-isolated) and classical keypair $(IK_{sk}^{DH}, IK_{pk}^{DH})$.

- **Symmetric roots:** Current root key $RK \in \{0,1\}^{32}$, live entropy pool $EP \in \{0,1\}^{32}$.
- **Directional chain keys:** Active sending chain key $CK_s \in \{0,1\}^{32}$ and counter $N_s \in \mathbb{N}$; active receiving chain key $CK_r \in \{0,1\}^{32}$ and counter $N_r \in \mathbb{N}$; previous receiving chain key CK_r^{prev} for in-flight message transition; previous chain length $PN_s \in \mathbb{N}$.
- **Classical DH ratchet state:** Local ephemeral keypair $(DH_{sk}^{loc}, DH_{pk}^{loc})$ and remote public key DH_{pk}^{rem}.
- **Asynchronous KEM buffer state:**
  - $\mathsf{pq_send_seq}, \mathsf{pq_recv_seq} \in \mathbb{N}$ (monotonic update counters).
  - $PQ^{\text{outstanding}}$: Map $\mathsf{kem_id} \mapsto (PQ_{sk}^{loc}, \mathsf{epoch}, \mathsf{timestamp})$ tracking local KEM proposals awaiting peer response.
  - $\mathsf{PQ}_{\text{staged}}$: Optional staged pending update tuple $(kem_id, SS_{PQ}, ct, RK_{\text{staged}}, \vec{CK}_{\text{staged}}, EP_{\text{staged}})$ awaiting commit confirmation.
  - PQ^{incoming}: Map $\mathsf{kem_id} \mapsto (SS_{PQ}, ct, \mathsf{seq})$ buffering completed responses for in-order application.
  - PQ^{retired}: Bounded FIFO set of retired $\mathsf{kem_id}$ values (capacity 256) preventing replay.
  - $\mathsf{pq_step_counter} \in \{0, \ldots, \Delta\}$ (ratchet interval countdown).
  - $\mathsf{transcript_hash} \in \{0,1\}^{32}$ (running hash of accepted ratchet updates).
- **Out-of-order skipped keys:** $\mathcal{K}_{skipped}$, a bounded cache (capacity $N_{\text{cache}} = 1{,}000$) mapping $(DH_{pk}^{rem}, N) \mapsto MK_N$.

A.2 Canonical Message Framing and Associated Data

Every transmitted packet $m_{\text{wire}} = (h, c)$ carries an unencrypted plaintext header h:

$$h = (\mathsf{sid},\ DH_{pk}^{loc},\ N_s,\ PN_s,\ \mathsf{flags},\ \mathsf{ext}).$$

The flag byte flags encodes the presence of asymmetric ratchet extensions:

- $\mathsf{FLAG_KEM_INIT} = \texttt{0x01}$: ext contains $(kem_id, PQ_{pk}^{loc}, \sigma_{\text{init}})$.
- $\mathsf{FLAG_KEM_RESP} = \texttt{0x02}$: ext contains $(kem_id, ct, \sigma_{\text{resp}})$.
- $\mathsf{FLAG_REGULAR} = \texttt{0x00}$: routine conversational message ($\mathsf{ext} = \varepsilon$).

The Associated Data AD_m for AES-256-GCM authenticated encryption is defined as:

$$AD_m \coloneqq \texttt{"StarMesh-Header-v1"} \,\|\, \mathsf{len}_{32}(h) \,\|\, h.$$

The ciphertext is computed as $c \xleftarrow{\$} \mathsf{AES\text{-}256\text{-}GCM\text{-}Enc}(MK_i, N_s^{32} \,\|\, \mathbf{0}^{64}, AD_m, m)$.

A.3 Canonical Asymmetric Update Signatures

The mutual authentication signatures σ_{init} and σ_{resp} are computed over canonical transcripts:

$$\begin{aligned}
m_{\text{init}} &\coloneqq \texttt{"UpdateAuth-Init"} \,\|\, \mathsf{sid} \,\|\, \mathsf{kem_id} \,\|\, PQ_{pk}^{loc} \,\|\, \mathsf{epoch} \,\|\, \mathsf{prev}, \\
\sigma_{\text{init}} &\xleftarrow{\$} \mathsf{ML\text{-}DSA.Sign}(IK_{sk,\text{init}}^{DSA},\, m_{\text{init}}), \\
m_{\text{resp}} &\coloneqq \texttt{"UpdateAuth-Resp"} \,\|\, \mathsf{sid} \,\|\, \mathsf{kem_id} \,\|\, PQ_{pk}^{loc} \,\|\, ct \,\|\, \mathsf{epoch}' \,\|\, \mathsf{prev}', \\
\sigma_{\text{resp}} &\xleftarrow{\$} \mathsf{ML\text{-}DSA.Sign}(IK_{sk,\text{resp}}^{DSA},\, m_{\text{resp}}),
\end{aligned}$$

where prev and prev' are the running transcript hashes of the respective last accepted updates.

A.4 Asynchronous KEM Buffer State Transitions and Canonical Tie-Breaking

Because ML-KEM shared secrets are not commutative and sequential HKDF evaluations are order-dependent, updates must be applied in a deterministic, synchronized order:

1. **Proposal Generation:** When $\mathsf{pq_step_counter} = 0$, the initiator generates $(PQ_{sk}^{loc}, PQ_{pk}^{loc}) \xleftarrow{\$} \mathsf{ML\text{-}KEM.KeyGen}(1^{\lambda})$, derives

$$\mathsf{kem_id} \leftarrow \mathsf{BLAKE3}(\texttt{"StarMesh-KEM-v1"} \,\|\, \mathsf{sid} \,\|\, PQ_{pk}^{loc} \,\|\, \mathsf{pq_send_seq}),$$

   records it in $PQ^{\text{outstanding}}$, and transmits with signature σ_{init}.

2. **Responder Staged Encapsulation and In-Flight Key Preservation:** The responder verifies σ_{init} and ensures $\mathsf{kem_id} \notin PQ^{\text{retired}}$. It computes $(ct, SS_{PQ}) \xleftarrow{\$} \mathsf{ML\text{-}KEM.Encaps}(PQ_{pk}^{loc})$, generates σ_{resp}, and evaluates the candidate staged derivation via a single extract and two expands:

$$\begin{aligned}
PRK_{\text{staged}} &\leftarrow \mathsf{HKDF\text{-}Extract}(salt = SS_{PQ},\ ikm = RK \,\|\, EP), \\
EP_{\text{staged}} &\leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\text{staged}},\ \texttt{"StarMesh-v1-EntropyPool"} \\
&\qquad \|\, \mathsf{kem_id} \,\|\, ct,\ 32), \\
(RK_{\text{staged}},\, \vec{CK}_{\text{staged}}) &\leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\text{staged}},\ \texttt{"StarMesh-v1-PQ-RK"} \\
&\qquad \|\, \mathsf{kem_id} \,\|\, ct,\ 96),
\end{aligned}$$

   where $\vec{CK}_{\text{staged}} = (CK_{A\to B}^{\text{staged}}, CK_{B\to A}^{\text{staged}})$. To prevent desynchronization if the reply is lost in transit, and to ensure initiator messages currently in flight under the existing chain remain decryptable, the responder does *not* immediately overwrite active root or chain keys. Instead, it buffers this state in $\mathsf{PQ}_{\text{staged}} \leftarrow (kem_id, SS_{PQ}, ct, RK_{\text{staged}}, \vec{CK}_{\text{staged}}, EP_{\text{staged}})$ and retains active CK_r. The responder attaches $(kem_id, ct, \sigma_{\text{resp}})$ to its reply.

3. **Responder Commit upon Peer Advance:** Upon receiving the first message or ACK authenticated under the new chain $CK_{A\to B}^{\text{staged}}$, the responder commits the staged state:

$$RK \leftarrow RK_{\text{staged}}, \quad CK_s \leftarrow CK_{B\to A}^{\text{staged}}, \quad EP \leftarrow EP_{\text{staged}},$$

promotes $CK_r \leftarrow CK_{A\to B}^{\text{staged}}$, retains the old receiving chain as CK_r^{prev} to handle any remaining out-of-order in-flight messages from the previous chain, clears $\mathsf{PQ}_{\text{staged}}$, zeroizes intermediate secrets, and resets $\mathsf{pq_step_counter} \leftarrow \Delta$.

4. **Initiator In-Order Draining:** The initiator verifies σ_{resp}, decapsulates $SS_{PQ} \leftarrow \mathsf{ML\text{-}KEM.Decaps}(PQ_{sk}^{loc}, ct)$, executes the identical derivation, sets $RK \leftarrow RK'$, $CK_s \leftarrow CK_{A\to B}$, $CK_r \leftarrow CK_{B\to A}$, $EP \leftarrow EP'$, zeroizes PQ_{sk}^{loc} and SS_{PQ}, retires $\mathsf{kem_id}$, and resets $\mathsf{pq_step_counter} \leftarrow \Delta$. The prior chains are retained as drained references until in-flight skips are committed.

5. **Concurrent Initiation and Canonical Lexicographical Tie-Break:** If both endpoints concurrently initiate proposals $U_A = (\mathsf{kem_id}_A, PQ_{pk,A})$ and $U_B = (\mathsf{kem_id}_B, PQ_{pk,B})$:

   - Both endpoints buffer the proposals and deterministically sort them by their authenticated 32-byte identifiers:
   $$U_{(1)} \coloneqq \arg\min_{U\in\{U_A,U_B\}} \mathsf{kem_id}(U), \quad U_{(2)} \coloneqq \arg\max_{U\in\{U_A,U_B\}} \mathsf{kem_id}(U),$$
   breaking ties by comparing identity keys IK_{pk}^{DSA}.
   - Both endpoints evaluate the updates in this identical canonical order: $U_{(1)}$ is applied to (RK, EP) to produce intermediate (RK_1, EP_1), and $U_{(2)}$ is applied chaining directly from (RK_1, EP_1) to produce (RK_2, EP_2) and active directional chain keys $\vec{CK}_2$.
   - Because the tie-break sort is canonical and deterministic, both endpoints arrive at the identical final root key and entropy pool regardless of network packet arrival order.

6. **Replay, Timeout, and Loss Recovery:** Any received $\mathsf{kem_id} \in PQ^{\text{retired}}$ is immediately discarded without updating state. If an outstanding proposal times out without reply, the initiator retires the identifier without changing RK and initiates a fresh proposal on retry. If the responder has a pending staged entry in $\mathsf{PQ}_{\text{staged}}$ and receives a retransmission or fresh proposal with a newer sequence number (indicating the previous reply was lost), the responder discards the uncommitted $\mathsf{PQ}_{\text{staged}}$ entry without modifying active RK, preventing permanent desynchronization.

A.5 Max-Skip Bound and Staged Commit-After-Authentication

To prevent denial-of-service and chain key destruction from unauthenticated packets:

1. **Max-Skip Threshold:** If $N - N_r > N_{\text{max_skip}} = 1{,}000$, the packet is immediately dropped.

2. **Staged Key Derivation:** The receiver creates a temporary scratch state $CK_{\text{temp}} \leftarrow CK_r$. It derives candidate skipped keys MK_j for $j \in \{N_r, \ldots, N-1\}$ and the target key MK_N without modifying live state.

3. **Verification and Conditional Commit:** The receiver evaluates
$$m \leftarrow \mathsf{AES\text{-}256\text{-}GCM\text{-}Dec}(MK_N,\ N^{32} \,\|\, \mathbf{0}^{64},\ AD_m,\ c).$$
   - If authentication succeeds: $\{MK_{N_r}, \ldots, MK_{N-1}\}$ are committed to $\mathcal{K}_{skipped}$, $N_r \leftarrow N+1$, and $CK_r \leftarrow CK_{\text{target}}$. MK_N is zeroized after decryption.
   - If authentication fails: the scratch buffer is immediately zeroized and discarded. Live CK_r, N_r, and $\mathcal{K}_{skipped}$ remain strictly unmodified.

References

[1] National Institute of Standards and Technology (NIST). *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. Federal Information Processing Standard, August 2024. https://doi.org/10.6028/NIST.FIPS.203

[2] National Institute of Standards and Technology (NIST). *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. Federal Information Processing Standard, August 2024. https://doi.org/10.6028/NIST.FIPS.204

[3] Signal Foundation. *The PQXDH Key Agreement Protocol*. Technical Report, 2023. https://signal.org/docs/specifications/pqxdh/ (Accessed: July 2026).

[4] G. Connell and R. Schmidt. *Signal Protocol and Post-Quantum Ratchets*. Signal Technology Foundation Blog, October 2, 2025. https://signal.org/blog/spqr/.

[5] Y. Dodis, D. Jost, S. Katsumata, T. Prest, and R. Schmidt. *Triple Ratchet: A Bandwidth-Efficient Hybrid-Secure Signal Protocol*. In *Advances in Cryptology – EUROCRYPT 2025*, Lecture Notes in Computer Science, Springer, 2025. IACR Cryptology ePrint Archive, Report 2025/078. https://eprint.iacr.org/2025/078

[6] B. Auerbach, Y. Dodis, D. Jost, S. Katsumata, and R. Schmidt. *How to Compare Bandwidth Constrained Two-Party Secure Messaging Protocols: A Quest for A More Efficient and Secure Post-Quantum Protocol*. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security 2025)*, USENIX Association, 2025. IACR Cryptology ePrint Archive, Report 2025/2267. https://eprint.iacr.org/2025/2267

[7] Apple Security Engineering and Architecture (SEAR). *iMessage with PQ3: The new state of the art in quantum-secure messaging at scale*. Apple Security Research Blog, February 2024. https://security.apple.com/blog/imessage-with-pq3/.

[8] M. Marlinspike and T. Perrin. *The X3DH Key Agreement Protocol*. Signal Foundation, 2016. https://signal.org/docs/specifications/x3dh/ (Accessed: July 2026).

[9] M. Marlinspike and T. Perrin. *The Double Ratchet Algorithm*. Signal Foundation, 2016. https://signal.org/docs/specifications/doubleratchet/ (Accessed: July 2026).

[10] J. Alwen, S. Coretti, and Y. Dodis. *The Double Ratchet: Security Notions, Proofs, and Modularization for the Signal Protocol*. In *Advances in Cryptology – EUROCRYPT 2019*, Lecture Notes in Computer Science, vol. 11476, pp. 129–158, Springer, 2019. https://doi.org/10.1007/978-3-030-17653-2_5

[11] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila. *A Formal Security Analysis of the Signal Messaging Protocol*. In *Proceedings of the 2nd IEEE European Symposium on Security and Privacy (EuroS&P 2017)*, pp. 451–466, IEEE, 2017. https://doi.org/10.1109/EuroSP.2017.27

[12] C. Cremers and M. Zhao. *Secure Messaging with Strong Compromise Resilience, Temporal Privacy, and Immediate Decryption*. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P 2024)*, pp. 2591–2609, IEEE, 2024. https://doi.org/10.1109/SP54263.2024.00038

[13] P. Maymounkov and D. Mazières. *Kademlia: A Peer-to-Peer Information System Based on the XOR Metric*. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS 2002)*, LNCS 2429, pp. 53–65, Springer, 2002. https://doi.org/10.1007/3-540-45748-8_5

[14] D. L. Chaum. *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms*. *Communications of the ACM*, 24(2):84–90, 1981. https://doi.org/10.1145/358549.358563

[15] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis. *The Loopix Anonymity System*. In *Proceedings of the 26th USENIX Security Symposium*, pp. 1199–1216, USENIX, 2017. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/piotrowska

[16] G. Danezis and I. Goldberg. *Sphinx: A Compact and Provably Secure Mix Format*. In *Proceedings of the 30th IEEE Symposium on Security and Privacy (S&P 2009)*, pp. 269–282, IEEE, 2009. https://doi.org/10.1109/SP.2009.15

[17] A. Langlois and D. Stehlé. *Worst-Case to Average-Case Reductions for Module Lattices*. *Designs, Codes and Cryptography*, 75(3):565–599, Springer, 2015. https://doi.org/10.1007/s10623-014-9938-4

[18] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. *CRYSTALS – Kyber: A CCA-Secure Module-Lattice-Based KEM*. In *Proceedings of the 3rd IEEE European Symposium on Security and Privacy (EuroS&P 2018)*, pp. 353–367, IEEE, 2018. https://doi.org/10.1109/EuroSP.2018.00032

[19] F. Giacon, F. Heuer, and B. Poettering. *KEM Combiners*. In *Proceedings of PKC 2018*, LNCS 10769, pp. 190–218, Springer, 2018. https://doi.org/10.1007/978-3-319-76578-5_7

[20] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila. *Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange*. In *Proceedings of PQCrypto 2019*, LNCS 11505, pp. 206–226, Springer, 2019. https://doi.org/10.1007/978-3-030-25510-7_12

[21] S. Blake-Wilson and A. Menezes. *Unknown Key-Share Attacks on the Station-to-Station (STS) Protocol*. In *Proceedings of PKC 1999*, LNCS 1560, pp. 154–170, Springer, 1999. https://doi.org/10.1007/3-540-49162-7_12

[22] M. Mosca. *Cybersecurity in an Era with Quantum Computers: Will We Be Ready?* *IEEE Security & Privacy*, 16(5):38–41, 2018. https://doi.org/10.1109/MSP.2018.3761723

[23] Session Foundation. *Session: Private Messenger Technical Overview*. https://docs.getsession.org/, 2023. (Accessed: July 2026).

[24] The Briar Project. *Briar Design Documentation*. https://briarproject.org/, 2023. (Accessed: July 2026).

[25] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon. *The Messaging Layer Security (MLS) Protocol*. IETF RFC 9420, July 2023. https://www.rfc-editor.org/rfc/rfc9420

[26] A. Langley, M. Hamburg, and S. Turner. *Elliptic Curves for Security*. IETF RFC 7748, January 2016. https://www.rfc-editor.org/rfc/rfc7748

[27] H. Krawczyk and P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. IETF RFC 5869, May 2010. https://www.rfc-editor.org/rfc/rfc5869

[28] H. Krawczyk. *Cryptographic Extraction and Key Derivation: The HKDF Scheme*. In *Proceedings of CRYPTO 2010*, LNCS 6223, pp. 631–648, Springer, 2010. https://doi.org/10.1007/978-3-642-14623-7_34

[29] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. *BLAKE3: One Function, Fast Everywhere*. Technical Report, 2020. https://github.com/BLAKE3-team/BLAKE3-specs (Accessed: July 2026).

[30] National Institute of Standards and Technology (NIST). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D, November 2007. https://doi.org/10.6028/NIST.SP.800-38D

[31] R. Canetti. *Universally Composable Security: A New Paradigm for Cryptographic Protocols*. In *Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science (FOCS 2001)*, pp. 136–145, IEEE, 2001. https://doi.org/10.1109/SFCS.2001.959888

[32] Y. Dodis, T. Ristenpart, and S. Vadhan. *Randomness Condensers for Efficiently Samplable, Seed-Dependent Sources*. In *Proceedings of TCC 2012*, LNCS 7194, pp. 618–635, Springer, 2012. https://doi.org/10.1007/978-3-642-28914-9_35

[33] B. Blanchet. *Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif*. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016. https://doi.org/10.1561/3300000004

[34] J. Jaeger and I. Stepanovs. *Optimal Channel Security Against Fine-Grained State Compromise: The Safety of Messaging*. In *Proceedings of CRYPTO 2018*, LNCS 10991, pp. 33–62, Springer, 2018. https://doi.org/10.1007/978-3-319-96878-0_2

[35] B. Poettering and P. Rösler. *Towards Bidirectional Ratcheted Key Exchange*. In *Proceedings of CRYPTO 2018*, LNCS 10991, pp. 3–32, Springer, 2018. https://doi.org/10.1007/978-3-319-96878-0_1

[36] B. Dowling, M. Fischlin, F. Günther, and D. Stebila. *A Cryptographic Analysis of the TLS 1.3 Handshake Protocol Candidates*. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS 2015)*, pp. 484–495, ACM, 2015. https://doi.org/10.1145/2810103.2813653

[37] J. Brendel, M. Fischlin, F. Günther, C. Janson, and D. Stebila. *Towards Post-Quantum Security for Signal's X3DH Handshake*. In *Proceedings of Selected Areas in Cryptography (SAC 2020)*, LNCS 12804, pp. 404–430, Springer, 2021. https://doi.org/10.1007/978-3-030-81652-0_16